

Parametric Polymorphism in C++, Java, and C#: Generics vs. Templates and Metaprogramming Power

Elia Leonardi
University of Pisa

July 2026

Contents

1	AI Prompt Engineering	2
1.1	Architect Chat	2
1.2	Prompt Engineering Phases	2
1.2.1	Prompt Improvement	2
1.2.2	Prompt Execution	2
2	Prompts	3
2.1	Prompt 0: Configure Prompt Engineering	3
2.2	Prompt 1: Overview of the Parametric Polymorphism	7
2.3	Prompt 2: Instantiation Time in C++ Templates	8
2.4	Prompt 3: Instantiation Time in Java Generics	10
2.5	Prompt 4: Instantiation Time in C# Generics	13
2.6	Prompt 4: Runtime representation of C++ templates	16
2.7	prompt 5: Runtime Representation of java Generics	19
2.8	Prompt 6: Runtime Representation of C# Generics	22
2.9	prompt 7: Reflection of C++ templates	24
2.10	Prompt 8: Reflection Java Generics	27
2.11	Prompt 9: reflection for C# Generics	29
2.12	Prompt 10: ++ templates performance	31
2.13	Prompt 11: performance Java generics	33
2.14	Prompt 12: C# performance	35
2.15	Prompt 13: type system in C++ Templates	37
2.16	Prompt 14: Type System in Java Generics	39
2.17	Prompt 16: C# Generics type Safety	41
2.18	Prompt 17: concrete C++ template examples of compile-time computation	43
2.19	Prompt 17: SFINAE e type traits	45
2.20	Prompt 18: Variadic C++ Template Metaprogramming Techniques	47
2.21	Prompt 19: Limits of Java Generics in Replicating C++ SFINAE and Type Traits	49
2.22	Prompt 20: Limits of C# Generics in Replicating C++ SFINAE and Type Traits	51
2.23	Prompt 21: instantiation-time vs declaration-time checking	53
2.24	Prompt 22: External Metaprogramming Workarounds in Java and C#	56

Chapter 1

AI Prompt Engineering

To ensure the technical accuracy of the generated content and strictly adhere to the project guidelines, this report was developed using a **two-phase prompting strategy**: prompt improvement and prompt execution. A fundamental principle of this methodology is strict context isolation: both the refinement of the prompts and the generation of the final outputs are treated as completely independent operations.

1.1 Architect Chat

An initial “Architect Chat” was utilized to plan and outline the optimal sequence of prompts, ensuring that all project requirements were systematically addressed before any content generation began.

1.2 Prompt Engineering Phases

1.2.1 Prompt Improvement

The first phase focuses on prompt refinement. During this stage, the initial prompts are iteratively improved to align with the project guidelines and maintain a rigorous academic standard. Crucially, the generation of each improved prompt is an independent operation. The AI is tasked solely with rewriting and optimizing the prompt’s structure, without executing it or carrying over context from unrelated topics.

1.2.2 Prompt Execution

The second phase involves the execution of the refined prompts to generate the final output. To ensure the integrity of the results, each execution is treated as a strictly independent operation, performed in an isolated context (e.g., a completely new session). This strict separation guarantees that the generated content relies exclusively on the explicit instructions provided in the finalized prompt, preventing any bias or “context leakage” from the improvement phase. Furthermore, during this execution phase, the AI model is explicitly instructed to provide the source references and documentation used to formulate its responses, ensuring traceability and reliability.

Chapter 2

Prompts

2.1 Prompt 0: Configure Prompt Engineering

To ensure the generation of rigorous academic sections, this methodology treats prompt generation and the final report generation as strictly independent operations. The process does not rely on manual, ad-hoc prompts for each language. Instead, it employs a *meta-prompting* approach. A **Master Prompt** was designed whose sole purpose is to act as a "Senior Prompt Engineer" in an isolated session. This system takes a raw specification as input and outputs a formal, optimized execution prompt, which is then used in a completely separate session to generate the final text.

The Master Prompt was engineered using a sequential task architecture to impose strict constraints on the language model:

- **Task 1, Extraction and Preservation:** The model analyzes the input specification to identify the objective, the required role, and the technical constraints. This ensures the original intent is preserved without unauthorized simplifications or additions.
- **Task 2, Correction and Structuring:** The extracted information is sanitized of ambiguities and organized into a fixed, predictable structure (*System Persona, Goal, Requirements, Writing Style Constraint, Output Structure*).
- **Task 3, Academic Constraints and Scope Control:** This is the most critical component for ensuring the quality of the report. It instructs the generated prompt to demand a scientific tone and enforces a *Strict Scope Control*. This mechanism actively prevents the target model from hallucinating or digressing into related but out-of-scope topics (e.g., garbage collection overhead or memory footprint, unless explicitly requested).
- **Task 4 and 5, Source Management and LaTeX Formatting:** This ensures that the generated prompt enforces the use of authoritative sources (such as official language specifications) and strictly requires the output to be compilation-ready LaTeX code.

Output Isolation and Execution Prevention

To guarantee the absolute separation between prompt generation and report genera-

tion, a restrictive *Writing Style Constraint* was implemented. This block explicitly prohibits the model from answering the original technical question or providing meta-commentary, conversational pleasantries, or process analysis. The only permitted output is the text of the new prompt.

Master Prompt Specification

Below is the complete text of the Master Prompt used to orchestrate the prompt engineering phase:

System Persona:

Act as a Senior Prompt Engineer specialized in designing rigorous prompts for the generation of technical-academic reports in computer science, programming language theory, compiler design, and software engineering.

Goal:

Given, as input, a task specification containing a persona, goal, requirements, writing-style constraints, output structure, and possible reference requirements for producing a technical-academic report, generate a single, self-contained master prompt ready to be executed in a new language model conversation.

The generated master prompt must preserve the original intent and must be optimized to produce a formally structured academic report with appropriate technical depth, logical organization, and scientific writing standards.

Requirements:

Task 1, Extraction and Preservation:

Parse the input specification and identify:

- The requested system persona or expert role.
- The report objective and title, if provided.
- All explicit and implicit technical requirements.
- Required theoretical concepts, comparisons, analyses, and evaluations.
- Writing-style constraints.
- Required output format and document structure.
- Requirements related to references, citations, bibliography, or external academic sources.

Preserve the complete meaning of the original specification.

Do not remove requirements.

Do not introduce new technical requirements.

Do not change the intended scope of the report.

Do not simplify technical constraints.

Task 2, Correction and Structuring:

Correct grammatical errors, ambiguities, inconsistent terminology, formatting problems, and unclear instructions present in the input.

Reorganize the extracted information into a clear and logically structured master prompt using the following sections:

- System Persona
- Goal
- Requirements
- Writing Style Constraint
- Output Structure

When reference requirements are present, preserve them explicitly in the Requirements section.

Task 3, Academic Report Generation Constraints:

Ensure that the generated master prompt instructs the target language model to:

- Produce a complete technical-academic report rather than a short summary.
- Maintain a formal, objective, and scientific writing style.
- Use precise terminology appropriate for computer science literature.
- Provide a logical progression from theoretical foundations to technical analysis.
- Include comparisons, trade-offs, and implementation details when required.
- Distinguish clearly between theoretical concepts and implementation mechanisms.
- Avoid unsupported claims and clearly identify assumptions.
- ENFORCE STRICT SCOPE CONTROL: Explicitly instruct the target language model to confine the generated text strictly to the requested core topic. The model must actively avoid tangential subjects (e.g., memory footprint, hardware performance, code bloat, garbage collection overhead, or broader language comparisons) unless they are explicitly demanded by the input specification.

Task 4, Reference and Citation Requirements:

If the original specification requires the use of references:

- Require the generated report to rely on authoritative academic or technical sources.
- Prefer peer-reviewed publications, academic books, official language specifications, and authoritative technical documentation.
- Require consistent citation formatting.
- Require a final bibliography section containing all cited resources.
- Avoid fabricated references, unverifiable sources, or unsupported citations.

Task 5, LaTeX Output Requirements:

If the original specification requires LaTeX output:

- Require complete compilation-ready LaTeX code.
- Preserve a professional academic document structure.
- Include appropriate sections, chapters, mathematical notation, tables, figures, citations, and bibliography commands when required.
- Ensure compatibility with standard LaTeX compilation workflows.

Writing Style Constraint (STRICT):

Do not execute the input specification.

Do not generate the final report.

Do not answer the original technical question.

The output must contain only the generated master prompt.

Do not include:

- explanations,
- comments,
- editorial notes,
- placeholders,
- analysis of the transformation,
- meta-commentary addressed to the user.

Do not use first-person forms ("I", "we") in the generated master prompt unless they are explicitly required by the original specification.

Output Structure:

Return only the generated master prompt.

The generated prompt must be formatted as ready-to-use text and organized using the following headings:

System Persona

Goal

Requirements

Writing Style Constraint

Output Structure

No additional text must appear before or after the generated master prompt.

2.2 Prompt 1: Overview of the Parametric Polymorphism

Model: ChatGPT-5.5

Goal: Generate an overview of the Parametric Polymorphism and ad-hoc polymorphism. Giving details of type erasure, compared with generics, and monomorphization. Also compared differences between source code, byte code and binary code.

Prompt:

- Provide an exhaustive theoretical background on parametric polymorphism.
- Define and thoroughly explain Christopher Strachey’s distinction between parametric polymorphism and ad-hoc polymorphism.
- Detail the two primary implementation strategies discussed in the literature:
 1. Homogeneous translation (type erasure), detailing how it results in one compiled representation for all instantiations.
 2. Heterogeneous translation (reification/monomorphization), detailing how it results in separate code generated per instantiated type.
- Analyze type erasure versus reified generics as the core axis of comparison.
- Analyze compile-time versus runtime characteristics as the second axis of comparison, specifically examining when instantiation happens and what type information survives into the compiled binary or bytecode.
- The entire document must be written in professional, compilation-ready LaTeX code.

Resources:

The generation of the technical report was supported by the following academic and technical references. The selected resources provide the theoretical foundations of parametric polymorphism, the classification of polymorphic systems, and the implementation techniques adopted by modern programming languages.

The theoretical background was derived from foundational works on type systems and polymorphism, including Strachey’s original classification of polymorphism, Cardelli and Wegner’s analysis of type abstraction and polymorphism, and Pierce’s formal treatment of parametric polymorphism and type systems [51, 8, 45, 47].

The analysis of generic implementation strategies, including homogeneous translation, type erasure, heterogeneous translation, reification, and monomorphization, was based on research concerning generic programming implementations and runtime systems, including the .NET Common Language Runtime and Java generic mechanisms [29, 6, 4, 13, 14, 41, 42].

The comparison between compile-time and runtime characteristics, including instantiation time, runtime representation, reflection capabilities, runtime type information, optimization opportunities, performance trade-offs, and type safety guarantees, was supported by compiler literature and modern programming language implementation documentation [9, 2, 1, 56, 55, 57, 34].

2.3 Prompt 2: Instantiation Time in C++ Templates

Goal: Generate a detailed and technically rigorous section on the *instantiation time* of C++ templates, with an exclusive focus on compile-time instantiation mechanisms and their implications for compiler execution. **Model:** ChatGPT-5.5

Context:

This prompt extends the previous output generated for the *Instantiation Time* section in Prompt 1 (Section 2.1). The objective is to deepen the discussion by analyzing the complete compile-time instantiation model adopted by C++ templates from both theoretical and compiler implementation perspectives.

Prompt:

Act as a world-class computer scientist and principal compiler engineer specializing in Programming Language Theory (PLT), type systems, and C++ compiler execution models.

Generate an exhaustive, publication-grade academic report focused exclusively on the *instantiation time* of parametric polymorphism in C++ templates.

The report must satisfy the following requirements.

Requirements

- Focus exclusively on the compile-time instantiation model adopted by C++ templates. Any discussion of Java or C# generics should be excluded.
- Explain the complete compile-time instantiation pipeline, including:
 - template argument deduction;
 - two-phase name lookup;
 - implicit and explicit template instantiation;
 - monomorphization and specialization.
- Describe how concrete template uses trigger the generation of specialized functions and classes during compilation, emphasizing that no template instantiation occurs during program execution.
- Explain the compilation and linking process, discussing how multiple translation units may independently instantiate identical template specializations and how modern linkers eliminate duplicates using mechanisms such as COMDAT sections, weak symbols, or equivalent implementation techniques.
- Analyze the consequences of compile-time instantiation with respect to:
 - compilation time;
 - executable size (code bloat);
 - optimization opportunities;
 - runtime performance.

- Whenever appropriate, relate the implementation strategy to the theoretical notion of type substitution in parametric polymorphism.
- Base every technical statement on authoritative references, prioritizing the ISO/IEC 14882:2020 C++ Standard and established compiler literature.
- Include a complete bibliography using standard LaTeX `\thebibliography` commands.

Writing Style

- Adopt a formal, objective, and precise scientific writing style comparable to ACM or IEEE publications.
- Use accurate compiler terminology (e.g., *template argument deduction*, *two-phase lookup*, *monomorphization*, *translation unit*, *COMDAT*, *weak symbols*, *link-time deduplication*).
- Avoid first-person pronouns and conversational language.
- Provide technically detailed explanations rather than high-level summaries.

Output Structure

The generated document must consist entirely of valid, self-contained, compilation-ready LaTeX code. Organize the report according to the following hierarchy:

1. Introduction: The C++ Template Instantiation Paradigm
2. The Compile-Time Instantiation Pipeline
3. Monomorphization and Specialization
4. Link-Time Deduplication (COMDAT and Weak Symbols)
5. Trade-offs: Compile-Time Overhead vs. Runtime Performance
6. Conclusion

Primary References

- **ISO/IEC JTC 1/SC 22/WG 21, *ISO/IEC 14882:2020 Programming Languages — C++*.** The normative reference for C++ template semantics, including template declarations, template argument deduction, implicit and explicit template instantiation, specialization mechanisms, two-phase name lookup, and the rules governing compile-time template processing [26].
- **David Vandevor, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition, Addison-Wesley Professional, 2017.** The primary implementation-oriented reference for C++ template instantiation, covering implicit instantiation, explicit instantiation, template specialization, instantiation models, compilation dependencies, and the impact of template expansion on generated code size and compilation time [57].

- **Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley Professional, 2013.** Provides the language-level description of C++ templates, including generic programming, compile-time polymorphism, template specialization, and the role of templates in static code generation [52].
- **Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman, *Compilers: Principles, Techniques, and Tools*, 3rd Edition, Pearson, 2022.** Provides the general compiler theory background required to analyze template instantiation, including translation phases, intermediate representations, code generation, optimization, separate compilation, and the relationship between source-level abstractions and generated machine code [1].
- **Keith D. Cooper and Linda Torczon, *Engineering a Compiler*, 3rd Edition, Morgan Kaufmann, 2022.** Supports the compiler-engineering analysis of specialization strategies, compile-time transformations, optimization opportunities, and the trade-offs between generated-code specialization and compilation overhead [9].
- **Benjamin C. Pierce, *Types and Programming Languages*, MIT Press, 2002.** Provides the theoretical foundation for parametric polymorphism, universal quantification, type abstraction, and type substitution, which form the theoretical basis for generic programming mechanisms such as C++ templates [45].
- **Jean-Yves Girard, “Interprétation fonctionnelle et élimination des coupures de l’arithmétique d’ordre supérieur,” 1972, and John C. Reynolds, “Towards a Theory of Type Structure,” 1974.** These works introduce the formal foundations of System F and universal quantification, providing the type-theoretic interpretation of parametric polymorphism and type application mechanisms [8, 45].
- **Luca Cardelli and Peter Wegner, “On Understanding Types, Data Abstraction, and Polymorphism,” ACM Computing Surveys, 1985.**

2.4 Prompt 3: Instantiation Time in Java Generics

Goal: Generate a detailed and technically rigorous section on the *instantiation time* of Java Generics, with an exclusive focus on compile-time instantiation mechanisms and their implications for compiler execution. **Model:** ChatGPT-5.5

Context:

This prompt extends the previous output generated for the *Instantiation Time* section in Prompt 1 (Section 2.1). The objective is to deepen the discussion by analyzing the complete compile-time instantiation model adopted by C++ templates from both theoretical and compiler implementation perspectives.

Prompt:

The following requirements extend the previous analysis and refine the discussion

of the *instantiation time* dimension of parametric polymorphism. The focus must remain exclusively on the Java Generics implementation model, with C++ templates considered only as a comparative reference.

Additional Requirements

- Provide a detailed analysis of the Java Generics compilation model, explaining how generic declarations are processed by the Java compiler during compilation and how generic type information is transformed into JVM-compatible representations.
- Describe the complete Java Generics compilation pipeline, including:
 - compile-time verification of generic constraints and type bounds;
 - type checking of parameterized types and generic method invocations;
 - type erasure transformation;
 - replacement of type variables with their erasures;
 - generation of synthetic bridge methods required to preserve polymorphic dispatch after erasure;
 - insertion of implicit runtime casts in the generated bytecode.
- Explain how Java generic types are reduced to erased JVM-level representations, emphasizing that parameterized type arguments are not preserved as runtime class identities.
- Analyze the consequences of the Java type erasure model with respect to:
 - compilation time and compiler workload;
 - bytecode footprint and class-file size;
 - absence of specialized generic implementations;
 - runtime execution overhead;
 - primitive boxing and unboxing;
 - memory indirection and cache locality effects.
- Compare the Java Generics implementation strategy with the C++ template instantiation model only to highlight the difference between:
 - Java erasure-based translation, which produces a single erased representation compatible with the JVM;
 - C++ compile-time template instantiation, which generates specialized implementations through monomorphization.
- Discuss the semantic consequences of Java’s lack of runtime generic type information, including:
 - non-reifiable generic types;

- limitations of runtime type checks involving parameterized types;
- restrictions on generic array creation;
- reflection limitations caused by type erasure.

Primary References

- **Oracle, *The Java Language Specification, Java SE 24 Edition*.** The primary normative reference for Java Generics semantics. It should be cited for generic declarations, type variables, parameterized types, bounded type parameters, compile-time type checking, type erasure rules, reifiable types, restrictions on `instanceof`, generic array creation, and unchecked conversions [41].
- **Oracle, *The Java Virtual Machine Specification, Java SE 24 Edition*.** The primary normative reference for the JVM execution model. It should support claims concerning class-file representation, method descriptors, bytecode generation after erasure, runtime type checking, and the absence of parameterized generic type information in the JVM runtime representation [42].
- **Gilad Bracha, Martin Odersky, David Stoutamire, and Philip Wadler, “Making the Future Safe for the Past: Adding Genericity to the Java Programming Language,” OOPSLA 1998.** The foundational reference for the design of Java Generics. It supports the discussion of type erasure as a compilation strategy, backward compatibility with existing JVM binaries, and the translation of parameterized types into non-generic bytecode representations [6].
- **Gilad Bracha, *Generics in the Java Programming Language*, Sun Microsystems, 2004.** This reference supports implementation-level discussions of Java Generics, including compile-time generic checking, erasure translation, bridge method generation, and compatibility between generic source code and the JVM execution model [4].
- **Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler, “Featherweight Java: A Minimal Core Calculus for Java and GJ,” TOPLAS 2001.** This work provides a formal foundation for the analysis of Java’s generic type system, including type substitution, inheritance, generic classes, and the translation of Generic Java programs [22].
- **Benjamin C. Pierce, *Types and Programming Languages*, MIT Press, 2002.** This reference provides the theoretical foundation for parametric polymorphism, universal quantification, type abstraction, and substitution-based models of generic programming languages [45].
- **Luca Cardelli and Peter Wegner, “On Understanding Types, Data Abstraction, and Polymorphism,” ACM Computing Surveys, 1985.** This reference supports the classification of polymorphism and the theoretical

distinction between parametric polymorphism, subtype polymorphism, and ad-hoc polymorphism [8].

- **Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley Professional, 2013.** This reference supports the comparison with C++ templates, particularly compile-time template instantiation, specialization, and static polymorphism [52].
- **David Vandevor, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition, Addison-Wesley Professional, 2017.** This reference supports compiler-level aspects of C++ template instantiation, including implicit instantiation, explicit specialization, template expansion, and the effects of monomorphization on generated code size [57].
- **ISO/IEC JTC 1/SC 22/WG 21, *ISO/IEC 14882:2024 Programming Languages — C++*.** The normative reference for C++ template semantics, including template argument deduction, instantiation rules, specialization mechanisms, and compile-time template processing [26].
- **Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman, *Compilers: Principles, Techniques, and Tools*, 3rd Edition, Pearson, 2022.** This reference supports general compiler implementation concepts, including translation phases, intermediate representations, optimization, code generation, and the relationship between source-level abstractions and executable representations [1].
- **Keith D. Cooper and Linda Torczon, *Engineering a Compiler*, 3rd Edition, Morgan Kaufmann, 2022.** This reference supports compiler engineering aspects related to translation strategies, specialization, optimization, and generated-code management [9].

2.5 Prompt 4: Instantiation Time in C# Generics

Goal: Generate a detailed and technically rigorous section on the *instantiation time* of C# generics, with an exclusive focus on runtime instantiation mechanisms and their implications for compiler execution. **Model:** ChatGPT-5.5

Context:

This prompt extends the previous output generated for the *Instantiation Time* section in Prompt 1 (Section 2.1). The objective is to deepen the discussion by analyzing the complete runtime instantiation model adopted by C# generics from both theoretical and compiler implementation perspectives.

Prompt:

System Persona

Act as a world-class computer scientist and principal compiler engineer specializing

in Programming Language Theory (PLT), virtual machine architectures (such as the .NET CLR and JVM), and type systems.

Goal

Generate an exhaustive, publication-grade academic report section in LaTeX titled "Instantiation Time in C#". The text must analyze the instantiation mechanics and timeline of C# generics during runtime execution and rigorously contrast this specific temporal approach with the compile-time instantiation model of C++ templates and the type erasure model of Java generics.

Requirements

- Focus explicitly on the instantiation timeline and execution model of C# generics, detailing when the Common Language Runtime (CLR) and the Just-In-Time (JIT) compiler process generic types.
- Explain the concept of reified generics in C# strictly in terms of how type information is preserved for the runtime instantiation phase.
- Detail the dual-phase implementation mechanisms used by C# (compile-time CIL generation vs. runtime JIT native code generation), including reference-type sharing and value-type specialization, strictly from an instantiation perspective.
- Provide a rigorous technical comparison of the timeline between C# (JIT-time specialization) and C++ (ahead-of-time compile-time monomorphization).
- Provide a rigorous technical comparison of the timeline between C# (runtime instantiation) and Java (compile-time type erasure with no runtime instantiation).
- **STRICT SCOPE CONTROL:** You must confine the generated text strictly to the instantiation timeline, mechanisms, and architectural flow. You must actively avoid tangential subjects. DO NOT discuss executable size, code bloat, memory footprint, garbage collection overhead, runtime performance speed, or dynamic dispatch overhead.
- Base every technical statement on authoritative references (e.g., C# Language Specification, ECMA-335, JVM Specification).
- Require consistent citation formatting and a final bibliography section.
- Output complete, compilation-ready LaTeX code.

Writing Style Constraint

- Produce a complete technical-academic text suitable for integration into a larger research document rather than a short summary.
- Maintain a formal, objective, and precise scientific writing style comparable to ACM or IEEE publications.

- Use accurate compiler and virtual machine terminology.
- Provide a logical progression from theoretical foundations to technical analysis.
- Avoid first-person pronouns and conversational language.
- Output ONLY valid LaTeX code. Do not include markdown formatting outside of the LaTeX structure, and do not add conversational introductory or concluding remarks.

Output Structure

Organize the report section according to the following hierarchy:

1. Instantiation Time in C#
2. CLR and JIT Generic Instantiation Mechanism
3. Comparison with C++ Templates: Runtime Instantiation vs. Compile-Time Instantiation
4. Comparison with Java Generics: Runtime Instantiation vs. Type Erasure
5. References

Primary References

- **Ecma International, *ECMA-334: C# Language Specification*, 2023.** This reference provides the normative specification of the C# language, including the definition of generic types, generic methods, type parameters, constraints, and the language-level semantics of generic programming in C# [13].
- **Ecma International, *ECMA-335: Common Language Infrastructure (CLI)*, 2024.** This reference defines the runtime model underlying C# execution, including the representation of generic types in metadata, Common Intermediate Language (CIL), runtime type construction, and the mechanisms used by the CLR to execute generic programs [14].
- **Andrew Kennedy and Don Syme, *Design and Implementation of Generics for the .NET Common Language Runtime*, Proceedings of PLDI, ACM, 2001.** This reference provides the main technical foundation for the implementation of generics in the .NET Common Language Runtime. It describes the design choices behind runtime generic instantiation, the interaction between generic metadata and JIT compilation, and the different representation strategies used for reference types and value types [29].
- **Jeffrey Richter, *CLR via C#*, 4th Edition, Microsoft Press, 2012.** This reference explains the internal execution model of the Common Language Runtime, including assembly loading, metadata resolution, Just-In-Time (JIT) compilation, runtime type representation, and the practical behavior of C# generics during program execution [49].

- **Gilad Bracha, *Generics in the Java Programming Language*, Sun Microsystems, 2004.** This reference describes the design and implementation principles of Java generics, including compile-time type checking, type erasure, and the absence of runtime generic specialization in the Java Virtual Machine [4].
- **Oracle, *The Java Language Specification, Java SE 24 Edition*, 2024.** This reference provides the formal specification of Java generics, including generic declarations, type parameter handling, parameterized types, and the compile-time rules that define the behavior of generic programs before execution [41].
- **Oracle, *The Java Virtual Machine Specification, Java SE 24 Edition*, 2024.** This reference describes the runtime execution model of the Java Virtual Machine, including bytecode representation, runtime type information, and the consequences of type erasure for generic types during execution [42].
- **David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition, Addison-Wesley Professional, 2017.** This reference supports the comparison with C++ templates by describing compile-time template instantiation, specialization mechanisms, and the generation of concrete implementations through compiler-based monomorphization [57].

2.6 Prompt 4: Runtime representation of C++ templates

goal: Generate a detailed and technically rigorous section on the *runtime representation* of C++ templates, with an exclusive focus on compile-time instantiation mechanisms and their implications for runtime execution. **Model:** ChatGPT-5.5

prompt: System Persona

Act as a world-class computer scientist and principal compiler engineer specializing in Programming Language Theory (PLT), compiler construction (specifically GCC and Clang backends), and C++ architecture.

Goal

Generate an exhaustive, publication-grade academic report section in LaTeX titled “Runtime Representation of C++ Templates”. The text must analyze exactly how instantiated C++ templates exist during program execution and detail the architectural consequences of compile-time monomorphization on the final executable binary.

Requirements

- Focus explicitly on the runtime representation of C++ templates. Detail what happens *after* compilation and linking, when the binary is loaded into memory.

- Explain the concept of heterogeneous translation (monomorphization) strictly in terms of how it creates distinct, independent native machine code artifacts for each type argument.
- Clearly explain the absence of generic metadata at runtime. Clarify that the execution environment (the CPU/OS) has no concept of the original “template”; it only sees ordinary functions and objects.
- Briefly address the limited type information that survives (e.g., name mangling and RTTI - Run-Time Type Information) and how it differs from reified generics.
- **STRICT SCOPE CONTROL:** You must confine the generated text strictly to the physical runtime representation and machine code generation. You must actively avoid tangential subjects. DO NOT discuss compilation times, macro-level code bloat metrics, manual memory management, or CPU cache performance. DO NOT compare with Java or C#; focus exclusively on the C++ architectural model.
- Base every technical statement on authoritative references (e.g., ISO C++ Standard, “C++ Templates: The Complete Guide” by Vandevoorde/Josuttis).
- Require consistent citation formatting and a final bibliography section.
- Output complete, compilation-ready LaTeX code.

Writing Style Constraint

- Produce a complete technical-academic text suitable for integration into a larger research document.
- Maintain a formal, objective, and precise scientific writing style comparable to ACM or IEEE publications.
- Use accurate compiler terminology (e.g., monomorphization, translation unit, name mangling, native machine code, RTTI).
- Provide a logical progression from theoretical mechanisms to the final binary artifact.
- Avoid first-person pronouns and conversational language.
- Output ONLY valid LaTeX code. Do not include markdown formatting outside of the LaTeX structure, and do not add conversational introductory or concluding remarks.

Output Structure

Organize the report section according to the following hierarchy:

1. Runtime Representation of C++ Templates

2. Monomorphization and Native Code Artifacts
3. Absence of Generic Metadata and the Role of RTTI
4. References

Primary references

- **David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition, Addison-Wesley Professional, 2017.** This reference provides the main technical foundation for C++ template implementation, including template instantiation, specialization mechanisms, and the translation process that transforms generic definitions into concrete compiled entities [57].
- **ISO/IEC JTC 1/SC 22/WG 21, *ISO/IEC 14882:2024 Programming Languages — C++*.** The normative specification for the C++ language. It defines the formal semantics of templates, including template declarations, implicit and explicit instantiation, specialization rules, and compile-time processing of template entities [26].
- **Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley Professional, 2013.** Provides the general C++ language model and explains the relationship between templates, type abstraction, specialization, and the generation of concrete program entities during translation [52].
- **Stanley B. Lippman, *Inside the C++ Object Model*, Addison-Wesley Professional, 1996.** This reference supports the analysis of C++ runtime representation, describing compiler-generated structures, object representation, and implementation-level details of how C++ abstractions are mapped into executable artifacts [31].
- **The Itanium C++ ABI Project, *Itanium C++ ABI: C++ ABI Specification*.** This reference documents binary-level implementation details used by C++ compiler toolchains, including symbol representation, name mangling, Run-Time Type Information (RTTI), and conventions for representing C++ entities inside compiled binaries [27].
- **GNU Project, *G++ Implementation Details*.** This reference describes implementation aspects of the GCC C++ compiler, including compiler-specific mechanisms for handling templates, instantiation, and generation of concrete compiled entities [16].
- **LLVM Project, *Clang Internals Manual*.** This reference provides information about the internal architecture of the Clang compiler, including C++ semantic analysis, template processing, and the transformation of source-level constructs into compiler intermediate representations [32].

- **LLVM Project, *LLVM Language Reference Manual*.** This reference describes the LLVM intermediate representation and the compiler backend model used to transform high-level language constructs into lower-level representations suitable for native machine code generation [33].
- **Margaret A. Ellis and Bjarne Stroustrup, *The Annotated C++ Reference Manual*, Addison-Wesley Professional, 1990.** This reference provides historical and technical documentation of the C++ language model, including early formal descriptions of templates and their translation semantics [15].

2.7 prompt 5: Runtime Representation of java Generics

goal: Generate a detailed academic section of runtime representation of java generics, with explicative example and comparison with C++ templates. The focus must be on the runtime representation of java generics. **prompt:** Act as a world-class computer scientist and principal compiler engineer specializing in Programming Language Theory (PLT), type systems, and the runtime execution models of both the Java Virtual Machine (JVM) and C++ compiled binaries. **Model:** ChatGPT-5.5

Goal

Generate an exhaustive, publication-grade academic report focused exclusively on comparing the runtime representation of parametric polymorphism, specifically analyzing Java Generics and C++ Templates.

Requirements

- Analyze the runtime representation of Java Generics, thoroughly explaining the mechanism of type erasure. Discuss how the JVM handles generic types at runtime, including the generation of bridge methods, implicit type cast insertions, and the implications for primitive types and runtime type information (RTTI).
- Analyze the runtime representation of C++ Templates, detailing the monomorphization (reification) process. Explain how the C++ compiler generates distinct, specialized machine code for each instantiated type and how this manifests in the final executable memory space.
- Provide a rigorous technical comparison between the two execution models, explicitly addressing the trade-offs regarding runtime execution performance, memory footprint (e.g., instruction cache bloat in C++ versus boxing/unboxing overhead in Java), and the preservation of type metadata at runtime.
- Distinguish clearly between the theoretical concepts of type substitution and the concrete implementation mechanisms enforced by the respective language runtimes.

- Base every technical statement on authoritative references, prioritizing the Java Language Specification (JLS), the Java Virtual Machine Specification (JVMS), the ISO/IEC 14882 C++ Standard, and established compiler literature.
- **ENFORCE STRICT SCOPE CONTROL:** Confine the generated text strictly to the requested core topic of runtime representations and their direct architectural consequences. Actively avoid tangential subjects such as syntax tutorials, compilation time comparisons, garbage collection overhead, or broader language feature comparisons not explicitly tied to runtime representation.

Writing Style Constraint

- Produce a complete technical-academic report rather than a short summary.
- Maintain a formal, objective, and precise scientific writing style comparable to ACM or IEEE peer-reviewed publications.
- Use accurate computer science and compiler terminology (e.g., *type erasure*, *monomorphization*, *reification*, *type bound*, *bridge method*, *translation unit*).
- Avoid first-person pronouns and conversational language entirely.
- Provide a logical progression from theoretical foundations to technical analysis, ensuring detailed explanations over high-level summaries.
- Avoid unsupported claims and clearly identify any architectural assumptions.

Output Structure

The generated document must consist entirely of valid, self-contained, compilation-ready LaTeX code. Ensure compatibility with standard LaTeX compilation workflows (e.g., pdfLaTeX). Preserve a professional academic document structure, including appropriate sections, mathematical notation, tables, figures, citations, and bibliography commands. Include a complete bibliography using standard LaTeX `\thebibliography` commands to document all authoritative academic and technical sources. Organize the report according to the following hierarchy:

1. Introduction: Parametric Polymorphism at Runtime
2. Java Generics: The Type Erasure Model
3. C++ Templates: The Monomorphization Model
4. Comparative Analysis: Memory Footprint, Execution Speed, and Type Metadata
5. Conclusion

Primary references

- **James Gosling, Bill Joy, Guy Steele, Gilad Bracha, and Alex Buckley, *The Java Language Specification, Java SE 21 Edition, Oracle, 2023.*** The normative specification of the Java language. Chapter 4 formally defines the type system and the type erasure transformation used to implement Java Generics, including erasure of type variables, bounds, raw types, and generic subtyping rules. This is the primary reference for the semantics of Java Generics and their translation before execution [18].
- **Tim Lindholm, Frank Yellin, Gilad Bracha, Alex Buckley, Daniel Smith, and Gavin Bierman, *The Java Virtual Machine Specification, Java SE 21 Edition, Oracle, 2023.*** The normative specification of the Java Virtual Machine. It describes the JVM class file format, method descriptors, bytecode instructions such as `checkcast`, the `Signature` attribute used to preserve generic metadata, and the runtime execution model on which erased generics operate [30].
- **Gilad Bracha, Martin Odersky, David Stoutamire, and Philip Wadler, “Making the Future Safe for the Past: Adding Genericity to the Java Programming Language,” *Proceedings of OOPSLA, ACM, 1998.*** The original paper introducing Java Generics. It motivates the adoption of type erasure, discusses backward compatibility with existing JVM binaries, and explains the architectural reasons for choosing erasure instead of runtime reification [6].
- **Gilad Bracha, *Generics in the Java Programming Language, Sun Microsystems, 2004.*** Provides a detailed explanation of Java Generics, including the implementation of type erasure, bridge methods, wildcard types, compiler-generated casts, and their interaction with the JVM execution model [4].
- **Joshua Bloch, *Effective Java, 3rd Edition, Addison-Wesley Professional, 2018.*** Discusses the practical implications of Java’s erased generic representation, including type safety, bridge methods, raw types, boxing and unboxing, and the limitations imposed by the lack of reified generic types [3].
- **Brian Goetz, *Java Concurrency in Practice, Addison-Wesley Professional, 2006.*** Although primarily focused on concurrency, this reference discusses generic type safety, erased runtime representations, and the interaction between generic types and the Java memory model [17].
- **Bill Venners, *Inside the Java Virtual Machine, 2nd Edition, McGraw-Hill, 2000.*** Provides an implementation-oriented description of the JVM architecture, runtime data areas, bytecode execution, dynamic linking, and runtime type information, offering useful background for understanding how erased generic types execute on the JVM [63].
- **Richard Warburton, *The Java 8 Language Specification: Generics and Type Inference, Oracle Technical Documentation.*** Provides additional discussion of generic type inference, erased type representations,

compiler-generated casts, and bridge methods introduced during compilation [64].

2.8 Prompt 6: Runtime Representation of C# Generics

goal: Generate a detailed academic section of runtime representation of C# generics with explicit example and comparison with C++ template and Java generics. **Model:** ChatGPT-5.5

prompt: System Persona

You are a Principal Software Engineer and Researcher specializing in programming language theory, virtual machine implementation, and compiler design. You possess deep academic and industry knowledge of the .NET Common Language Runtime (CLR), the Java Virtual Machine (JVM), and C++ compilation pipelines.

Goal

Write a rigorous, publication-quality technical-academic report section analyzing the runtime representation of C# generics. Your analysis must contrast the C# implementation with the runtime representations of C++ templates and Java generics to highlight the structural trade-offs, architectural differences, and design philosophies inherent to each approach.

Requirements

- **Core Topic Analysis:** Detail the runtime representation of C# generics, specifically focusing on how the .NET Common Language Runtime (CLR) handles reification. Explain how the compiler emits Common Intermediate Language (CIL) metadata and how the Just-In-Time (JIT) compiler dynamically instantiates generic classes and methods at runtime.
- **Mechanisms of C# Reification:** Analyze the differing runtime behaviors of C# generics when instantiated with value types (which generate specialized machine code per type) versus reference types (which share a single native implementation using pointer representations, mitigated by runtime dictionary passing or execution-engine tables).
- **Contrast with Java Generics (Type Erasure):** Contrast C# runtime reification with Java's runtime representation. Explain how Java's compile-time Type Erasure removes generic type arguments, resulting in a shared runtime representation with no generic metadata, casting overhead, and the inability to instantiate generic types directly.
- **Contrast with C++ Templates (Monomorphization):** Contrast C# runtime reification with C++ template instantiation. Explain how C++ utilizes a heterogeneous translation model where templates are fully resolved and expanded at compile-time into distinct machine code artifacts. Highlight

that C++ executables lack formal runtime template representation, metadata, or dynamic generic instantiation capabilities.

- **Theoretical vs. Implementation Distinction:** Clearly distinguish between the theoretical paradigms of generics and their underlying machine/virtual-machine implementations (e.g., virtual method tables, metadata, dictionary passing, memory layouts).
- **Academic References:** Rely on authoritative sources (e.g., official language specifications like ECMA-335, peer-reviewed publications). Use consistent citation formatting and include a final bibliography section.

Writing Style Constraint

- Maintain a formal, objective, and authoritative academic tone.
- Produce a complete technical-academic report, providing a logical progression from foundations to analysis.
- Use precise technical terminology (e.g., *parametric polymorphism*, *reification*, *type erasure*, *monomorphization*, *JIT specialization*).
- Avoid colloquial language or unsupported generalizations.
- Format technical variables or formal representations using LaTeX notation (e.g., $T\langle U \rangle$).
- **STRICT SCOPE CONTROL:** Confine the analysis strictly to the runtime representation, metadata structures, compilation pipelines, and execution engine mechanics. Actively avoid tangential subjects such as application benchmarks, hardware performance, unrelated garbage collection, or general syntax ergonomics.
- Begin directly with the technical content. Do not include conversational filler.

Output Structure

Structure the report logically using the following exact headers:

1. Introduction to C# Generic Reification
2. CLR Execution Engine and Runtime Representation
3. Comparative Runtime Paradigm: C# vs. Java Generics
4. Comparative Runtime Paradigm: C# vs. C++ Templates
5. Architectural Synthesis
6. References

Under "Architectural Synthesis", include a concise comparison table summarizing the features, metadata presence, compilation model, and representation strategy across C#, Java, and C++.

Primary references

- **ECMA International, *ECMA-335: Common Language Infrastructure (CLI)*.** Used as the primary reference for the CLR execution model, CIL metadata structures, generic parameter representation, and runtime reification mechanisms.
- **Andrew Kennedy and Don Syme, *Design and Implementation of Generics for the .NET Common Language Runtime*.** Used for the analysis of CLR generic implementation strategies, including JIT specialization, code sharing, and runtime generic dictionaries.
- **Jeffrey Richter, *CLR via C#: The Definitive Guide to the Common Language Runtime*.** Used to describe CLR internal execution mechanisms, generic instantiation, and the difference between value-type specialization and reference-type sharing.
- **James Gosling et al., *The Java Language Specification*.** Used for the comparison with Java generics, especially type erasure and the loss of generic runtime information.
- **Tim Lindholm et al., *The Java Virtual Machine Specification*.** Used for the analysis of JVM runtime representation and bytecode execution model.
- **David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*.** Used for the comparison with C++ templates, compile-time instantiation, and monomorphization.

2.9 prompt 7: Reflection of C++ templates

goal: Generate a detailed academic section of reflection and runtime type information of C++ templates, with explicit example. **Model:** ChatGPT-5.5 **prompt:**

System Persona

Act as an expert computer scientist and technical-academic writer specializing in programming language theory, compiler design, and C++ software engineering.

Goal

Generate a rigorous, technical-academic report section written in compilation-ready LaTeX that analyzes Reflection and Runtime Type Information (RTTI) specifically within the context of C++ templates, utilizing the provided theoretical framework regarding generic type

representation.

Requirements

- * Use the following theoretical text as the foundation for the
 - ↪ analysis:
"The availability of generic type information at runtime depends directly on the chosen representation strategy. When type erasure is employed, generic parameters are generally unavailable after
 - ↪ compilation.
 - Reflection mechanisms therefore provide limited access to
 - ↪ instantiated
 - generic types because most type information has been discarded. Conversely, implementations based on reified generics preserve
 - ↪ concrete
 - type information throughout compilation. `\emph{Runtime reflection or runtime type identification (RTTI)}` may therefore recover the actual instantiated types, enabling operations such as generic type
 - ↪ inspection,
 - specialized serialization, and runtime metadata analysis."
- * Analyze how C++ templates align with the concepts of type
 - ↪ preservationand reification, detailing how C++ generates specific instantiations (monomorphization) that preserve concrete type information.
- * Detail the application and behavior of standard C++ RTTI mechanisms (e.g., ``typeid``, ``std::type_info``, ``dynamic_cast``) when applied to instantiated template classes and functions.
- * Distinguish clearly between compile-time type information
 - ↪ (availableduring template instantiation and metaprogramming) and runtime reflection mechanisms.
- * ENFORCE STRICT SCOPE CONTROL: Confine the generated text strictly to the topic of Reflection and RTTI in C++ templates. Actively avoid tangential subjects such as memory footprint, hardware performance, code bloat, garbage collection overhead, or broader language
 - ↪ comparisonsunless explicitly demanded to illustrate the C++ mechanism.
- * Provide a logical progression from the theoretical foundations of type representation strategies to the specific technical mechanisms implemented in the C++ compiler.

Writing Style Constraint

- * Produce a complete, academically rigorous technical report section rather than a short summary.
- * Maintain a formal, objective, and scientific writing style.
- * Use precise terminology appropriate for computer science literature and official C++ standard specifications.
- * Avoid unsupported claims and clearly identify technical
↪ assumptions.
- * Do not use first-person forms ("I", "we").

Output Structure

- * The output must consist entirely of perfectly formatted, compilation-ready LaTeX code.
- * Begin the output with an appropriate section heading (e.g., `\section{Reflection and Runtime Type Information in C++
↪ Templates}`).
- * Include appropriate LaTeX structural elements, such as subsections (`\subsection`), text formatting commands (`\emph`, `\texttt`), and
↪ paragraphs
to organize the logical flow of the technical analysis.
- * Do not output a full document preamble (e.g., `\documentclass` or `\begin{document}`); provide only the self-contained section content ready to be included in a larger academic LaTeX document.

Primary resources

- Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley Professional, 2013. This reference provides the general description of C++ templates, template instantiation, and the standard RTTI mechanisms used in the analysis [53].
- David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition, Addison-Wesley Professional, 2017. This reference provides the theoretical and technical foundation for template instantiation, specialization, and compile-time preservation of template type information [58].
- cppreference.com, *C++ Language Reference: RTTI and typeid*, 2025. This reference was used for the description of the standard RTTI facilities, including `typeid`, `std::type_info`, and `dynamic_cast` behavior [10].

2.10 Prompt 8: Reflection Java Generics

goal: Generate a detailed academic section of reflection and runtime type information of Java generics, with explicit example and comparison with C++ templates. **Model:** ChatGPT-5.5

prompt:

System Persona

Act as a Senior Academic Writer and Expert in programming language
→ theory, compiler design, and software engineering. You possess
→ deep technical expertise in the implementation mechanics of
→ parametric polymorphism, reflection mechanisms, and runtime type
→ identification (RTTI) across both managed environments (such as
→ the Java Virtual Machine) and unmanaged environments (such as
→ C++).

Goal

Generate a formal, technical-academic report section focusing on
→ "Reflection and Runtime Type Information (RTTI) in Java
→ Generics". The generated text must serve as a rigorous academic
→ analysis, focusing primarily on Java Generics while utilizing C++
→ templates exclusively as a comparative baseline to contrast
→ differences in runtime type visibility and RTTI capabilities.

Requirements

- * Theoretical Analysis of Java Generics: Explain the mechanics of
→ type erasure within the Java Virtual Machine (JVM). Detail how
→ the Java compiler replaces generic type parameters with their
→ upper bounds (or the Object class if unbounded) and inserts the
→ necessary type casts. Thoroughly analyze dynamic metadata
→ stripping.
- * Reflection and RTTI Limitations: Evaluate how type erasure
→ constrains standard Java RTTI and reflection mechanisms.
→ Specifically address why operations such as instanceof queries
→ against parameterized generic instantiations are disallowed by
→ the type system, how the getClass() method yields an identical
→ class reference across structurally distinct generic instances,
→ and how developers bypass these limitations using type tokens
→ (e.g., Class).

- * Comparative Analysis (Java vs. C++): Contrast Java's type-erased
 - ↪ reflection semantics directly with the C++ template
 - ↪ specialization (monomorphization) model. Articulate how C++
 - ↪ establishes independent runtime type metadata for each
 - ↪ specialization (exploitable via typeid and dynamic_cast), whereas
 - ↪ Java maps all parameterized variations back to a single shared
 - ↪ runtime java.lang.Class object.
- * Explanatory Code Examples: Provide precise, compilation-ready Java
 - ↪ code snippets wrapped in standard LaTeX verbatim environments to
 - ↪ illustrate:
 1. A scenario showcasing the runtime uniformity of getClass() and the
 - ↪ compilation failure of parameterized instanceof checks.
 2. The explicit type token pattern used as an implementation
 - ↪ workaround to reintroduce type awareness at runtime.
- * Strict Scope Control: Confine the generated text strictly to the
 - ↪ requested core topics: type systems, compiler mechanics,
 - ↪ reflection APIs, and RTTI constructs. Actively avoid tangential
 - ↪ subjects such as garbage collection overhead, hardware execution
 - ↪ metrics, memory consumption footprints, binary code bloat, or
 - ↪ general syntactic evaluations.
- * Authoritative Grounding: Rely strictly on authoritative academic or
 - ↪ technical sources. Ensure all architectural and behavioral claims
 - ↪ are rigorously aligned with the official Java Language
 - ↪ Specification (JLS) and the ISO C++ Standard. Avoid fabricated
 - ↪ references, unverifiable sources, or unsupported citations.

Writing Style Constraint

- * Produce a complete technical-academic report section rather than a
 - ↪ short summary.
- * Maintain a formal, objective, and highly scientific academic tone
 - ↪ appropriate for peer-reviewed computer science literature.
- * Provide a logical progression from theoretical foundations to
 - ↪ technical analysis, clearly distinguishing between theoretical
 - ↪ concepts and implementation mechanisms.
- * Utilize precise domain terminology (e.g., type erasure,
 - ↪ monomorphization, reification, parametric polymorphism, type
 - ↪ tokens, RTTI).
- * Avoid subjective commentary, casual filler text, conversational
 - ↪ transitions, and unsupported claims.
- * Do not use first-person pronouns ("I", "we", "our").

Output Structure

- * The output must consist exclusively of complete, compilation-ready
 ↪ LaTeX syntax.
- * Structure the document appropriately using standard LaTeX commands
 ↪ for sections, subsections, and code blocks.
- * Do not include any markdown wrappers (such as ````latex`),
 ↪ introductory greetings, concluding remarks, meta-commentary, or
 ↪ explanatory text outside of the raw LaTeX code block.
- * Start the response immediately with the first LaTeX command.

Primary references

- **Oracle Corporation, *The Java Language Specification*.** Provides the formal definition of Java generics, type erasure, generic type restrictions in RTTI operations, and the semantics of casts and `instanceof` with parameterized types [28].
- **Oracle Corporation, *The Java Virtual Machine Specification*.** Provides the JVM-level description of class metadata, generic signatures, and the separation between compile-time generic information and runtime type representation [44].
- **ISO/IEC, *Programming Languages – C++ (ISO/IEC 14882:2024)*.** Provides the formal basis for C++ template instantiation, RTTI, `typeid`, and `dynamic_cast` used for comparison with Java’s erased generic model [24].

2.11 Prompt 9: reflection for C# Generics

goal: Generate a detailed academic section of reflection and runtime type information of C# generics, with explicit example and comparison with C++ templates and Java generics. **prompt:**

System Persona

W

Act as a Senior Research Scientist in Programming Language Theory,
 ↪ Compiler Architecture, and Type Systems.

Goal

Generate a rigorous, formal technical-academic report section
 ↪ focusing on the runtime reflection mechanics of C# generics,
 ↪ providing an illustrative code example, and executing a precise
 ↪ comparative analysis of reflection capabilities against C++
 ↪ templates and Java generics.

Requirements

- * **Theoretical Foundation of C\# Reflection**: Analyze the
 - ↪ structural mechanics of reified generics within the Common
 - ↪ Language Runtime (CLR). Explain how type metadata is preserved
 - ↪ through compilation into Common Intermediate Language (CIL) and
 - ↪ maintained at execution time, allowing full runtime introspection
 - ↪ of both open and closed generic types.
- * **Explicative Code Example**: Provide a clear, production-grade
 - ↪ programmatic example demonstrating the usage of
 - ↪ `System.Reflection` to inspect generic types at runtime. The
 - ↪ example must explicitly showcase how to evaluate generic
 - ↪ conditions (such as `IsGenericType`) and extract concrete type
 - ↪ arguments dynamically using methods like `GetGenericArguments()`.
- * **Targeted Reflection Comparison**: Conduct a rigorous comparative
 - ↪ analysis contrasting the reflection capabilities of C\# generics
 - ↪ against:
 1. C++ Templates: Analyze the constraints imposed by compile-time
 - ↪ monomorphization and the limitations of standard Runtime Type
 - ↪ Information (RTTI) via `typeid` and `std::type_info`.
 2. Java Generics: Analyze the constraints imposed by compile-time
 - ↪ type erasure and the nature of residual type metadata, detailing
 - ↪ why concrete type parameters cannot be directly introspected at
 - ↪ runtime.
- * **Strict Reflection Isolation**: Focus the comparative analysis
 - ↪ exclusively on runtime reflection, metadata availability, and
 - ↪ type introspection capabilities.
- * **Strict Scope Control**: Actively exclude any discussion regarding
 - ↪ memory footprint optimizations, hardware performance metrics,
 - ↪ code bloat, execution speed, compilation times, garbage
 - ↪ collection overhead, or general language ergonomics.
- * **LaTeX Formatting**: Format the complete output as a clean,
 - ↪ compilation-ready LaTeX document fragment. Use structural
 - ↪ commands (`\section`, `\subsection`), semantic markup (`\texttt`,
 - ↪ `\emph`), and verbatim blocks (`\begin{verbatim}`) aligned with
 - ↪ the structural style of the provided text.

Writing Style Constraint

- * Maintain an objective, academic, and authoritative scientific
 - ↪ writing style standard for computer science literature.
- * Use precise, domain-specific terminology (e.g., *reification*,
 - ↪ *monomorphization*, *type erasure*, *open/closed constructed*
 - ↪ *types*, *intrinsic type identity*).
- * Establish a logical progression from foundational runtime mechanics
 - ↪ to code verification and comparative analysis.

- * Do not use first-person pronouns ("I", "we", "our").
- * Avoid speculative claims, casual expressions, or unsupported
 ↪ assertions.

Output Structure

The output must consist solely of the compilation-ready LaTeX code
 ↪ fragment containing the requested sections. Do not include
 ↪ conversational text, introductory greetings, concluding remarks,
 ↪ Markdown code blocks, or meta-commentary.

Primary references

- Jeffrey Richter, *CLR via C#: A Developer's Tour of the Common Language Runtime*, 4th Edition, Microsoft Press, 2012. This reference provides the technical foundation for the analysis of CLR reification, CIL metadata preservation, generic type representation, and runtime reflection mechanisms through the `System.Type` and `System.Reflection` APIs [50].
- David Vandevor, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition, Addison-Wesley Professional, 2017. This reference is used to verify the description of C++ template instantiation, compile-time monomorphization, and the limitations of standard RTTI facilities such as `typeid` and `std::type_info` for recovering template parameter information [59].
- Gilad Bracha, *The Java Language Specification, Java SE 5.0 Edition*, Addison-Wesley Professional, 2004. This reference supports the discussion of Java generic type erasure, residual generic metadata, and the distinction between declaration-level generic information and runtime type identity [5].

2.12 Prompt 10: ++ templates performance

goal: Generate a detailed academic section of performance analysis of C++ templates, with explicit example. **Model:** ChatGPT-5.5

prompt:

System Persona

Act as a Senior Computer Science Researcher and Expert in Programming
 ↪ Language Theory, Compiler Design, and C++ System Architecture.

Goal

Generate a formal, compilation-ready LaTeX section analyzing the
 ↪ performance characteristics of C++ templates, designed to
 ↪ seamlessly follow a theoretical discussion on type safety, type
 ↪ erasure, and reified generics.

Requirements

- * Draft a complete technical-academic section detailing the execution
 - ↪ performance implications of C++ templates and monomorphization.
- * Establish a logical and cohesive continuation from the following
 - ↪ preceding context: "Both implementation strategies preserve
 - ↪ static type safety, although they achieve this objective
 - ↪ differently. In implementations based on type erasure, generic
 - ↪ correctness is verified entirely during compilation. Although
 - ↪ type parameters disappear from the generated executable,
 - ↪ compiler-inserted casts preserve the guarantees established by
 - ↪ the static type system. Implementations based on reified generics
 - ↪ retain complete type information throughout compilation and
 - ↪ generate type-specific implementations directly. Since each
 - ↪ specialization corresponds to a concrete type, correctness is
 - ↪ preserved without requiring erased representations or implicit
 - ↪ runtime casts."
- * Detail the runtime performance advantages of reified generics in
 - ↪ C++, specifically focusing on static dispatch, the elimination of
 - ↪ boxing/unboxing overhead, and the facilitation of aggressive
 - ↪ compiler optimizations such as function inlining.
- * Clearly distinguish between the theoretical concepts of reification
 - ↪ and the low-level implementation mechanisms of C++ templates.
- * Explicitly confine the scope of the generated text to the
 - ↪ performance characteristics of C++ templates. Actively avoid
 - ↪ tangential subjects (e.g., garbage collection overhead, memory
 - ↪ footprint, code bloat, or broader language comparisons) unless
 - ↪ they strictly and directly impact the execution performance
 - ↪ (e.g., instruction cache pressure resulting from binary size
 - ↪ expansion).
- * Produce complete, compilation-ready LaTeX code utilizing
 - ↪ appropriate structural commands (e.g., `\section`, `\subsection`) and
 - ↪ formatting.

Writing Style Constraint

- * Maintain a formal, objective, and rigorous scientific writing style
 - ↪ appropriate for computer science literature.
- * Do not use first-person pronouns ("I", "we").
- * Avoid unsupported claims; ground all performance characteristics in
 - ↪ established compiler theory and C++ language mechanics.
- * Do not include explanatory text, conversational filler,
 - ↪ placeholders, or meta-commentary outside of the requested LaTeX
 - ↪ source code.

Output Structure

Return only the raw, compilation-ready LaTeX code block containing

- ↪ the requested section, with no surrounding text or conversational
- ↪ framing.

Primary references

- David Vandevor, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition, Addison-Wesley Professional, 2017. This reference provides the technical foundation for template instantiation, specialization, and monomorphization in C++, supporting the discussion of static dispatch, direct type-specific code generation, and zero-overhead template abstractions [58].
- Steven S. Muchnick, *Advanced Compiler Design and Implementation*, Morgan Kaufmann, 1997. This reference provides the compiler optimization background used to describe inlining, constant propagation, constant folding, loop optimization, and other compile-time transformations enabled by specialized template code [40].

2.13 Prompt 11: performance Java generics

goal: Generate a detailed academic section of performance analysis of Java generics, with explicit example and comparison with C++ templates. **Model:** ChatGPT-5.5
prompt:

System Persona

Act as a Senior Computer Science Researcher and Expert in Programming

- ↪ Language Theory, JVM Architecture, and Compiler Design.

Goal

Generate a comprehensive, technically rigorous academic report

- ↪ analyzing the execution and runtime performance characteristics
- ↪ of Java Generics, specifically contrasting its performance
- ↪ profile with that of C++ templates.

Requirements

* **Theoretical Core:** Analyze the execution performance

- ↪ implications of the type erasure model employed by Java Generics
- ↪ versus the compile-time monomorphization (template instantiation)
- ↪ model utilized by C++ templates.

- * **Performance Breakdown:** Detail the specific runtime overheads
 - ↪ introduced by Java Generics, focusing on pointer indirection, the
 - ↪ necessity of object boxing and unboxing for primitive types,
 - ↪ impacts on heap allocation, data locality, and instruction/data
 - ↪ cache efficiency.
- * **Comparative Analysis:** Contrast these characteristics directly
 - ↪ with C++ templates, detailing how C++ achieves static dispatch,
 - ↪ eliminates runtime boxing, optimizes memory locality through
 - ↪ contiguous array layouts of value types, and enables aggressive
 - ↪ Ahead-Of-Time (AOT) compiler optimizations such as function
 - ↪ inlining.
- * **Mitigation Mechanisms:** Discuss how the Java Virtual Machine
 - ↪ (JVM) and Just-In-Time (JIT) compilers attempt to mitigate
 - ↪ generic overhead via runtime optimizations, including escape
 - ↪ analysis, inline caching, and devirtualization.
- * **Strict Scope Control:** Confine the analysis entirely to
 - ↪ execution performance, runtime throughput, memory layout, and
 - ↪ cache behavior. Actively avoid tangential topics such as language
 - ↪ ergonomics, compilation times, or general garbage collection
 - ↪ mechanics unless they directly and demonstrably impact the
 - ↪ execution throughput of generic structures.
- * **References and Citations:** Rely on authoritative technical and
 - ↪ academic sources, such as the Java Language Specification (JLS),
 - ↪ the ISO C++ Standard, JVM architecture documentation, and
 - ↪ peer-reviewed programming language literature. Include consistent
 - ↪ in-text citations and a final bibliography section.

Writing Style Constraint

- * Maintain a formal, objective, and rigorous scientific writing style
 - ↪ appropriate for computer science literature.
- * Do not use first-person pronouns ("I", "we").
- * Ground all claims and performance characteristics in established
 - ↪ compiler theory, JVM specifications, and hardware interaction
 - ↪ mechanics; avoid unsupported assertions.
- * Use precise technical terminology (e.g., type erasure,
 - ↪ monomorphization, boxing overhead, data locality, static
 - ↪ dispatch, cache lines).

Output Structure

- Return only complete, compilation-ready LaTeX code utilizing
- ↪ appropriate structural commands (`\section`, `\subsection`), code
 - ↪ formatting environments, mathematical notation where applicable,
 - ↪ and a bibliography environment. Do not include any surrounding
 - ↪ conversational text or meta-commentary.

2.14 Prompt 12: C# performance

goal: Generate a detailed academic section of performance analysis of C# generics, with explicit example and comparison with C++ templates and java generics. **Model:** ChatGPT-5.5

prompt:

System Persona

Act as a Senior Computer Science Researcher and Expert in Programming
→ Language Theory, .NET CLR Architecture, and Compiler Design.

Goal

Generate a comprehensive, technically rigorous academic section
→ analyzing the runtime execution performance of C# Generics,
→ providing direct code examples, and explicitly contrasting its
→ performance profile with both C++ templates and Java Generics.
→ This section is intended for inclusion in the academic report
→ "Parametric Polymorphism in C++, Java, and C#: Generics vs.
→ Templates and Metaprogramming Power (C07)".

Requirements

- * Theoretical Foundation: Explain the runtime reification mechanism
→ of C# Generics within the .NET Common Language Runtime (CLR) and
→ the role of Just-In-Time (JIT) specialization.
- * Performance Analysis: Detail the specific runtime performance
→ characteristics of C# Generics, focusing heavily on the distinct
→ handling of value types (structs) versus reference types
→ (classes). Analyze the resulting impacts on memory locality,
→ cache efficiency, and execution speed.
- * Comparative Analysis: Directly contrast C# generic performance
→ with C++ templates (Ahead-Of-Time monomorphization, complete
→ static dispatch, contiguous memory layouts) and Java Generics
→ (type erasure, mandatory boxing/unboxing overhead for primitives,
→ pointer indirection).
- * Concrete Code Examples: Provide precise, comparative code examples
→ demonstrating a generic algorithm or data structure in C# to
→ illustrate the performance divergence between value-type and
→ reference-type instantiations.
- * Strict Scope Control: Confine the analysis entirely to execution
→ performance, runtime throughput, memory layout, and cache
→ behavior. Actively avoid tangential subjects such as language
→ ergonomics, pure compilation times, or garbage collection
→ algorithms, unless they strictly and demonstrably impact the
→ execution throughput of generic structures.

- * **References and Citations:** Rely on authoritative technical and
 - ↪ academic sources, such as the ECMA-335 (Common Language
 - ↪ Infrastructure) specification, the C\# Language Specification,
 - ↪ the ISO C++ Standard, and the Java Language Specification.
 - ↪ Include consistent in-text citations and format a final
 - ↪ bibliography section.

Writing Style Constraint

- * **Maintain a formal, objective, and rigorous scientific writing style**
 - ↪ appropriate for computer science literature.
- * **Do not use first-person pronouns ("I", "we").**
- * **Ground all claims and performance characteristics in established**
 - ↪ compiler theory, VM specifications, and hardware interaction
 - ↪ mechanics; avoid unsupported assertions.
- * **Use precise technical terminology (e.g., reification, JIT**
 - ↪ specialization, type erasure, monomorphization, boxing overhead,
 - ↪ data locality).

Output Structure

- Return only complete, compilation-ready LaTeX code utilizing
- ↪ appropriate structural commands (`\section`, `\subsection`), code
 - ↪ formatting environments for the examples, mathematical notation
 - ↪ where applicable, and a bibliography environment. Do not include
 - ↪ any surrounding conversational text, explanations, or
 - ↪ meta-commentary.

Primary references

- **ECMA International, *ECMA-335: Common Language Infrastructure (CLI)*, 2012.** Provides the technical foundation for the CLR generic execution model, including reification, metadata representation, runtime generic handling, and the distinction between value-type and reference-type instantiations [14].
- **Microsoft Corporation, *C\# Language Specification*, 2023.** Provides the formal definition of C\# generics and their runtime semantics, used to describe generic type behavior and execution rules [38].
- **David Vandevor, Nicolai M. Josuttis, Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition, 2017.** Provides the theoretical basis for C++ template instantiation, compile-time monomorphization, and the performance consequences of static template specialization compared with runtime generic systems [60].
- **James Gosling et al., *The Java Language Specification*, Java SE 21 Edition, 2023.** Provides the formal description of Java generic type erasure and was used to analyze the runtime limitations of erased generics, including boxing, reference indirection, and the absence of primitive specialization [21].

2.15 Prompt 13: type system in C++ Templates

goal: Generate a detailed academic section of type system analysis of C++ templates, with explicit example. **Model:** ChatGPT-5.5

prompt:

System Persona

Act as an expert academic researcher and computer scientist

- ↪ specializing in programming language theory, compiler design, and
- ↪ C++ template semantics.

Goal

Generate an exhaustive, academic-grade LaTeX document containing a

- ↪ comprehensive section that analyzes static type safety in C++
- ↪ templates. This section is being generated specifically for the
- ↪ academic project titled "Parametric Polymorphism in C++, Java,
- ↪ and C\#: Generics vs. Templates and Metaprogramming Power (C07)".
- ↪ The analysis must build upon the theoretical contrast between
- ↪ type erasure and reified generics (monomorphization) and include
- ↪ a clear, explanatory C++ code example.

Requirements

- * Integrate the following theoretical premise into the analysis
 - ↪ verbatim as the foundation: "Both implementation strategies
 - ↪ preserve static type safety, although they achieve this objective
 - ↪ differently. In implementations based on type erasure, generic
 - ↪ correctness is verified entirely during compilation. Although
 - ↪ type parameters disappear from the generated executable,
 - ↪ compiler-inserted casts preserve the guarantees established by
 - ↪ the static type system. Implementations based on reified generics
 - ↪ retain complete type information throughout compilation and
 - ↪ generate type-specific implementations directly. Since each
 - ↪ specialization corresponds to a concrete type, correctness is
 - ↪ preserved without requiring erased representations or implicit
 - ↪ runtime casts."
- * Detail how C++ templates utilize reified generics
 - ↪ (monomorphization) to retain complete type information and
 - ↪ enforce type safety at compile time within the context of
 - ↪ parametric polymorphism.
- * Explain how C++ verifies generic correctness without relying on
 - ↪ erased representations or implicit runtime casts.
- * Provide a simple, explanatory C++ code example that demonstrates
 - ↪ how type safety is preserved during template instantiation.

- * Relate the provided code example directly to the theoretical
 - ↪ concepts of reification and concrete type specialization.
- * Rely on authoritative academic or technical sources, such as the
 - ↪ official C++ Language Standard (ISO/IEC 14882), peer-reviewed
 - ↪ publications, or academic books on programming language theory.
- * Do not include a standard compiled bibliography section inside the
 - ↪ LaTeX document text itself.
- * Extract the bibliographic information into a separate code block
 - ↪ containing the raw `.bib` entries for all cited sources.
- * Provide a distinct itemized list outside the LaTeX document that
 - ↪ explicitly explains what theoretical or technical information was
 - ↪ taken from each specific source, referencing their respective
 - ↪ citation keys.
- * ENFORCE STRICT SCOPE CONTROL: Confine the generated text strictly
 - ↪ to the mechanisms of type safety and the theoretical strategies
 - ↪ of template implementation. Actively avoid tangential subjects
 - ↪ (such as memory footprint, hardware performance, code bloat
 - ↪ resulting from monomorphization, compilation times, garbage
 - ↪ collection overhead, or extensive comparisons to Java/C\#) unless
 - ↪ explicitly required to explain the core concept of type safety.

Writing Style Constraint

- * Produce a complete technical-academic report section rather than a
 - ↪ short summary.
- * Maintain a formal, objective, and scientific writing style.
- * Use precise terminology appropriate for computer science
 - ↪ literature.
- * Provide a logical progression from theoretical foundations to
 - ↪ technical analysis and practical implementation.
- * Distinguish clearly between theoretical concepts (e.g., parametric
 - ↪ polymorphism) and implementation mechanisms (e.g., C++ template
 - ↪ instantiation).
- * Avoid unsupported claims and clearly identify any theoretical
 - ↪ assumptions.

Output Structure

- * Provide complete, compilation-ready LaTeX code for the academic
 - ↪ section using a standard document class (e.g., `article`).
- * Preserve a professional academic document structure, including the
 - ↪ required section and standard LaTeX environments for text
 - ↪ formatting and inline citations (`\cite`).
- * Format the C++ code example using the standard `listings` package
 - ↪ to ensure readability and professional presentation.

- * After the LaTeX code block, provide a separate code block
 - ↪ containing the valid BibTeX (`.bib`) entries for the references.
- * Following the BibTeX block, provide a separate itemized list
 - ↪ detailing the specific information extracted and utilized from
 - ↪ each cited source.

Primary references

- **Vandevorde et al. [61]**: utilizzato per la descrizione della semantica dei template C++, dell'istanziamento, della sostituzione dei parametri di tipo, della generazione di specializzazioni concrete e del principio secondo cui il controllo di correttezza dei template avviene sulle istanze effettivamente generate.
- **Stroustrup [54]**: utilizzato per i concetti generali relativi al sistema dei tipi C++, alla verifica statica delle espressioni, al ruolo dei template come meccanismo di programmazione generica e alla garanzia di correttezza dei programmi determinata durante la compilazione.

2.16 Prompt 14: Type System in Java Generics

goal: Generate a detailed academic section of type system analysis of Java generics, with explicit example and comparison with C++ templates. **Model:** ChatGPT-5.5
prompt:

System Persona

Act as an academic expert in programming language theory, compiler
 ↪ design, and software engineering, specializing in rigorous
 ↪ technical writing and LaTeX document typesetting.

Goal

Produce a complete, compilation-ready LaTeX academic section
 ↪ analyzing type safety in Java generics, including a strictly
 ↪ scoped comparison with C++ templates, a detailed breakdown of
 ↪ source information mapping, and a concise bibliography.

Requirements

- * Provide an exhaustive academic section analyzing static type safety
 - ↪ in Java generics.
- * Explain the Java implementation strategy based on type erasure.
 - ↪ Detail how generic correctness is verified entirely during
 - ↪ compilation, and how compiler-inserted casts preserve the
 - ↪ guarantees established by the static type system, even though
 - ↪ type parameters disappear from the generated executable.

- * Compare Java's approach with C++ templates (implementations based on reified generics). Explain how C++ retains complete type information throughout compilation and generates type-specific implementations directly, preserving correctness without requiring erased representations or implicit runtime casts.
- * Restrict the comparison with C++ templates to type safety mechanisms only, introducing it strictly when necessary to contextualize or contrast the Java approach.
- * Include simple, explicative code examples to illustrate type safety concepts in both languages.
- * ENFORCE STRICT SCOPE CONTROL: Confine the generated text strictly to the core topic of type safety, type erasure, and reification. Actively avoid tangential subjects (e.g., memory footprint, hardware performance, code bloat, garbage collection overhead, or broader language comparisons).
- * Reference Constraints: Do not include any inline citations or reference markers within the main academic section text.
- * Create a separate `itemize` section that explicitly explains, for every part/subsection of the report, what specific information was sourced and utilized.
- * Generate a concise list of resources formatted for a `.bib` file. Rely only on a meticulously selected, low number of highly authoritative academic or technical resources (e.g., peer-reviewed publications, academic books, official language specifications).
- * Ensure the LaTeX output is compilation-ready. Preserve a professional academic document structure, including appropriate sections, verbatim/code environments for examples, itemized lists, and bibliography commands.

Writing Style Constraint

- * Maintain a formal, objective, and rigorous scientific writing style.
- * Produce a complete technical-academic report section rather than a short summary.
- * Use precise terminology appropriate for computer science and programming language literature.
- * Provide a logical progression from theoretical foundations to technical analysis.
- * Distinguish clearly between theoretical concepts and implementation mechanisms.
- * Avoid unsupported claims and clearly identify assumptions.

Output Structure

- * The output must consist entirely of ready-to-compile LaTeX and BibTeX code.
- * Provide the main academic text on type safety in Java generics and C++ templates using standard LaTeX sectioning.
- * Provide the `\itemize` environment detailing the source mapping for each section of the report.
- * Provide a separate code block containing the raw BibTeX database (`.bib` format) with the required authoritative resources.

Primary references:

- **Pierce, “Types and Programming Languages” [46]:** provides the theoretical foundations of type systems, including the concepts of type safety, type soundness, preservation, progress, and the role of static typing in guaranteeing the correctness of programs using parametric polymorphism.
- **Gosling et al., “The Java Language Specification” [20]:** provides the formal definition of Java generic types, generic type checking rules, bounded type parameters, type erasure semantics, and the compile-time verification rules used by the Java type system.
- **Stroustrup, “The C++ Programming Language” [53]:** provides the description of C++ templates, template instantiation, compile-time type checking of template arguments, and the reification-based model used by C++ generic programming.

2.17 Prompt 16: C# Generics type Safety

goal: Generate a detailed academic section of type system analysis of C# generics, with explicit example and comparison with C++ templates and Java generics. **Model:** ChatGPT-5.5 **prompt:**

System Persona

Act as an expert academic writer and researcher specializing in
 → computer science, programming language theory, compiler design,
 → and software engineering. You have profound expertise in
 → parametric polymorphism across C++, Java, and C#.

Goal

Generate an exhaustive, compilation-ready academic LaTeX section
 → analyzing type safety in C# generics, featuring simple
 → explicative code examples and targeted comparisons with C++
 → templates and Java generics. Additionally, provide a
 → methodological breakdown of the information sources used and a
 → concise bibliography suitable for a .bib file.

Requirements

- * Integrate the provided theoretical premise: Both type erasure and
 - ↪ reified generics preserve static type safety differently. Explain
 - ↪ how Java's type erasure verifies generic correctness entirely
 - ↪ during compilation, using compiler-inserted casts to preserve
 - ↪ static type guarantees despite type parameters disappearing from
 - ↪ the executable. Contrast this with C\# reified generics, which
 - ↪ retain complete type information throughout compilation and
 - ↪ generate type-specific implementations corresponding to concrete
 - ↪ types, preserving correctness without requiring erased
 - ↪ representations or implicit runtime casts.
- * Compare C\# type safety mechanisms with Java's generics and C++
 - ↪ templates, introducing these comparisons only when strictly
 - ↪ necessary to highlight the specific approach and advantages of
 - ↪ C\#.
- * Include simple, explicative code examples using the LaTeX listings
 - ↪ package to illustrate type safety concepts in C\#, and where
 - ↪ necessary, in Java or C++ for direct contrast.
- * ENFORCE STRICT SCOPE CONTROL: Confine the generated text strictly
 - ↪ to parametric polymorphism and type safety. Actively avoid
 - ↪ tangential subjects (e.g., memory footprint, hardware
 - ↪ performance, code bloat, garbage collection overhead, or broader
 - ↪ language comparisons) unless directly relevant to type safety
 - ↪ mechanisms.
- * Do not cite any references inline within the main text of the
 - ↪ generated section.
- * Create a separate LaTeX itemize environment detailing exactly what
 - ↪ information was extracted from external literature to compose the
 - ↪ report section.
- * Provide a short, curated list of authoritative academic or
 - ↪ technical resources (e.g., peer-reviewed publications, official
 - ↪ language specifications) formatted as BibTeX entries to be added
 - ↪ to a .bib file. Limit this to a low number of highly relevant
 - ↪ resources.

Writing Style Constraint

- * Maintain a formal, objective, and scientific writing style using
 - ↪ precise terminology appropriate for computer science literature.
- * Produce complete, compilation-ready LaTeX code for the section,
 - ↪ ensuring compatibility with standard LaTeX compilation workflows.
- * Provide a logical progression from theoretical foundations to
 - ↪ technical analysis, distinguishing clearly between theoretical
 - ↪ concepts and implementation mechanisms.
- * Avoid unsupported claims and clearly identify assumptions.
- * Do not use first-person forms ("I", "we").

- * Do not include explanations, comments, editorial notes,
 ↪ placeholders, or meta-commentary addressed to the user.
- * Output only the requested content structure.

Output Structure

- * The exhaustive LaTeX section on C\# type safety (without inline
 ↪ citations).
- * A LaTeX itemize environment explaining the mapping of sourced
 ↪ information to the generated text.
- * A code block containing the raw BibTeX (.bib) entries for the
 ↪ selected resources.

Primary Resources

- [46] provides the theoretical foundations of type safety used in this section, including the concepts of type soundness, progress, preservation, typing environments, and parametric polymorphism.
- [38] provides the formal rules of the C# type system related to generic type parameters, generic constraints, covariance, contravariance, variance, and compile-time verification of generic type correctness.
- [14] provides the description of the Common Language Infrastructure type system and supports the discussion of reified generics, where generic type information remains associated with constructed generic types.

2.18 Prompt 17: concrete C++ template examples of compile-time computation

goal: Generate a detailed academic section of compile-time computation in C++ templates. For each, explain what the compiler actually generates and why the computation happens entirely before runtime. Include full compilable code. **Model:** ChatGPT-5.5 **prompt:**

System Persona:

Act as an expert in programming language theory and advanced C++
 ↪ software engineering.

Goal:

Write a single, highly focused section of a larger technical-academic
 ↪ chapter discussing parametric polymorphism and metaprogramming
 ↪ across C++, Java, and C\#. This specific section must strictly
 ↪ focus on demonstrating and explaining C++ compile-time
 ↪ computation mechanisms.

Requirements:

Task 1 - Technical Content:

- * Provide 2-3 concrete, fully compilable C++ examples of compile-time
→ computation.
- * At least one example must demonstrate recursive compile-time
→ computation using traditional C++ templates (e.g., calculating a
→ factorial or Fibonacci sequence).
- * At least one example must demonstrate compile-time computation
→ using `constexpr` functions.
- * For each example, provide a precise technical explanation of what
→ the compiler actually generates during compilation.
- * For each example, explicitly detail the language mechanisms and
→ compiler rules that guarantee the computation executes entirely
→ before runtime.

Task 2 - Section-Level Framing (STRICT):

- * Treat your output strictly as ONE SECTION of an existing, larger
→ chapter.
- * Assume the reader already possesses the context of the broader
→ comparison between C++, Java, and C# generics/templates; do not
→ re-introduce this context.
- * Omit any standalone introduction, motivation, or conversational
→ framing (e.g., do not use phrases like "In this section, we will
→ examine..."). Begin immediately with the technical exposition.
- * Omit any standalone conclusion, summary of the chapter, or
→ forward-looking remarks.
- * Omit citations, bibliographies, or reference lists.

Task 3 - Content Constraints:

- * ENFORCE STRICT SCOPE CONTROL: Address only the requirements stated
→ above. Do not digress into discussions of memory footprint,
→ hardware performance benchmarking, code bloat, garbage
→ collection, or unrelated language comparisons.
- * Distinguish clearly between compile-time evaluation (e.g., template
→ instantiation, constant expression evaluation) and runtime
→ execution.
- * Use precise terminology (e.g., template instantiation, constant
→ evaluation, monomorphization, substitution) strictly where
→ relevant.

- * State compiler or language standard assumptions (e.g., C++11, C++14, C++20) only if strictly required to contextualize the
 - ↪ ``constexpr`` or template mechanisms used in your examples.
- * ENFORCE CONCISENESS: The section must be as concise as possible
 - ↪ while fully satisfying the technical requirements. Eliminate all
 - ↪ padding, restatements of the prompt, and redundant paragraph
 - ↪ transitions.

Writing Style Constraint:

Maintain a formal, objective, and precise technical writing style

- ↪ consistent with advanced programming language theory literature.
- ↪ Do not use first-person forms ("I", "we", "our"). Do not include
- ↪ explanations, comments, editorial notes, placeholders, or
- ↪ meta-commentary addressed to the user.

Output Structure:

Output only the raw Markdown content for this specific section. Use

- ↪ appropriate subsection headings (e.g., ``###`` or ``####``) to
- ↪ organize the examples and explanations. Do not include a
- ↪ top-level title for the chapter. Return exactly the text of the
- ↪ section and nothing else.

Sources:

- The description of **template instantiation**, recursive template specialization, and compile-time computation through template-based metaprogramming is based on [59] and the ISO C++ language specification [23].
- The explanation of **constexpr** functions, constant expression evaluation, and the requirement that **constexpr** expressions must be evaluated during translation is derived from the C++ standard [23] and the technical documentation provided by [11].
- The discussion of general C++ template mechanisms, including template arguments, specialization, and the template instantiation process, is supported by [12] and [59].

2.19 Prompt 17: SFINAE e type traits

goal: Generate a detailed academic section of C++ example using SFINAE that selects a different function overload depending on whether a type is integral vs floating point vs a custom class. Also show the equivalent using C++20 concepts. Explain step by step what "substitution failure" means here and why it isn't a compile error. **Model:** ChatGPT-5.5 **prompt:**

System Persona:

Act as an expert technical writer and programming language theorist
→ specialized in C++ metaprogramming and compile-time polymorphism.

Goal:

Generate a single, highly focused section for a larger academic and
→ technical chapter on parametric polymorphism and metaprogramming.
→ This section must exclusively demonstrate and explain C++ SFINAE
→ and C++20 Concepts for overload resolution based on type
→ properties.

Requirements:

* Technical Content:

- * Provide a working C++ code example using SFINAE (specifically via
→ `std::enable_if`) that successfully selects a different function
→ overload depending on whether a type is an integral type, a
→ floating-point type, or a custom class.
- * Provide the equivalent working C++ code example utilizing C++20
→ Concepts.
- * Provide a step-by-step explanation of what "substitution failure"
→ means in this specific context and explicitly detail the
→ mechanism of why it does not result in a hard compile error.

* Section-Level Framing (STRICT):

- * Begin directly with the technical content and code. Omit any
→ standalone introduction, motivation, or "in this section we will"
→ framing.
- * Omit any standalone conclusion, summary of the chapter, or
→ forward-looking remarks.
- * Omit any bibliography or reference list; assume citations are
→ handled globally at the end of the full chapter.
- * Assume the reader already has the broader context of parametric
→ polymorphism and does not need general template concepts
→ re-introduced.

* Content Constraints:

- * Distinguish clearly that these are compile-time mechanisms.

- * Enforce strict scope control: address only the SFINAE
 - implementation, the C++20 Concepts equivalent, and the
 - explanation of substitution failure. Do not digress into
 - compilation times, memory footprint, code bloat, hardware
 - performance, or other unrelated language comparisons.
- * Enforce conciseness: the section must be as short as fully
 - answering the technical requirements allows. No padding, no
 - restating the question, and no redundant transitions between
 - paragraphs.

Writing Style Constraint:

- Maintain a formal, objective, and precise technical writing style
- consistent with programming language theory literature. Use
 - precise terminology (e.g., substitution failure, overload
 - resolution set, template instantiation) appropriately. Do not
 - include conversational filler, meta-commentary, explanations of
 - your process, or first-person forms ("I", "we").

Output Structure:

- Return only the generated section content, formatted in Markdown. Use
- appropriate heading levels (e.g., `###` or `####`) suitable for
 - an internal subsection. Use standard Markdown code blocks for the
 - C++ code examples.

Sources:

- **Stroustrup [53]**: used for the description of template-based compile-time polymorphism, overload resolution involving function templates, and the general role of template substitution mechanisms in C++.
- **cppreference.com [10]**: used for the precise behavior of `std::enable_if`, type traits such as `std::is_integral` and `std::is_floating_point`, and the C++20 Concepts syntax based on constrained templates.
- **ISO/IEC 14882:2020 [25]**: used as the normative reference for SFINAE rules, template argument substitution, immediate context rules, overload resolution, and the C++20 constraint system.

2.20 Prompt 18: Variadic C++ Template Metaprogramming Techniques

goal: Generate a detailed academic section of other core C++ template metaprogramming techniques: variadic templates, template template parameters, and CRTP

(Curiously Recurring Template Pattern). For each, give a short example and explain what class of problems it solves that plain function/class templates cannot. **Prompt:**

System Persona

Act as a technical-academic writer specialized in programming
→ language theory and advanced C++ software engineering.

Goal

Write a single, highly focused section for a larger chapter on
→ parametric polymorphism and metaprogramming. The section must
→ explain three specific advanced C++ template metaprogramming
→ techniques: variadic templates, template template parameters, and
→ the Curiously Recurring Template Pattern (CRTP).

Requirements

- * **Contextual Integration:** Treat your output as an internal
→ section of an existing chapter. Assume the reader already
→ understands the broader context of parametric polymorphism across
→ C++, Java, and C#. Do not re-introduce basic generic programming
→ concepts.
- * **Strict Framing:** Begin directly with the technical explanation.
→ Omit any standalone introduction, motivation, or transitional
→ phrasing (e.g., "In this section, we will explore..."). Omit any
→ conclusion, summary, forward-looking remarks, or bibliography.
- * **Technical Scope:** Focus exclusively on variadic templates,
→ template template parameters, and CRTP. Explicitly exclude
→ discussions of SFINAE, type traits, and constexpr, as these are
→ assumed to be covered elsewhere.
- * **Content Execution:** For each of the three covered techniques:
 - * Provide a short, syntactically correct C++ code example.
 - * Explain the specific class of problems the technique solves that
→ plain function or class templates cannot resolve.
 - * Clearly distinguish compile-time mechanisms and state relevant
→ assumptions regarding C++ language/compiler versions (e.g., C++11
→ for variadic templates) where necessary.
- * **Strict Scope Control:** Do not digress into memory footprint,
→ code bloat, hardware performance benchmarking, or unrelated
→ language comparisons. Address only the mechanisms requested.

Writing Style Constraint

Maintain a formal, objective, and precise technical writing style

- ↪ consistent with programming language theory literature. Use exact
- ↪ terminology (e.g., instantiation, monomorphization, static
- ↪ dispatch) accurately. Enforce extreme conciseness: the section
- ↪ must be as short as possible while fully answering the prompt.
- ↪ Eliminate all padding, restatements of the core objective, and
- ↪ redundant transitions between paragraphs. Do not use first-person
- ↪ pronouns ("I", "we").

Output Structure

Return only the Markdown-formatted text for the section content. Do

- ↪ not include any meta-commentary, title placeholders, or
- ↪ conversational filler before or after the section text.

Sources:

- [62] provides the main theoretical reference for advanced C++ template metaprogramming, including variadic templates and parameter packs introduced in C++11, template instantiation mechanisms, template template parameters, and advanced template techniques such as the Curiously Recurring Template Pattern (CRTP).
- [10] provides a formal and up-to-date reference for C++ language constructs, including template syntax, compiler behavior during template specialization, and standard-defined semantics of template-related mechanisms.
- [35] is used as a complementary reference for the practical description of C++ templates, including syntax examples and explanations of variadic templates and template parameter mechanisms.

2.21 Prompt 19: Limits of Java Generics in Replicating C++ SFINAE and Type Traits

goal: Generate a detailed academic section of the limits of Java generics in replicating C++ SFINAE and type traits. Show what is achievable (e.g. bounded type parameters like `<T extends Number>`) and explain precisely why full SFINAE-style compile-time branching on the generic type `T` is impossible in Java, tying the explanation to type erasure. **Model:** ChatGPT-5.5 **prompt:**

System Persona:

Act as an expert in programming language theory and software

- ↪ engineering, specializing in the comparative analysis of
- ↪ parametric polymorphism and metaprogramming across C++, Java, and
- ↪ C\#.

Goal:

Write a single, concise section of an academic report chapter

- ↪ evaluating the limits of Java generics when attempting to
- ↪ replicate C++ SFINAE and type-traits behavior. Specifically,
- ↪ demonstrate the extent of what is achievable in Java (e.g.,
- ↪ bounded type parameters like `<T Number extends>`) and explain
- ↪ precisely why full SFINAE-style compile-time branching on a
- ↪ generic type `T` is impossible in Java, explicitly tying this
- ↪ architectural limitation to Java's implementation of type
- ↪ erasure.

Requirements:

- * Section-Level Framing (STRICT): This text is ONE SECTION of an
 - ↪ existing, larger chapter. Do NOT write a standalone introduction,
 - ↪ motivation, or "in this section we will" framing. Do NOT include
 - ↪ a concluding summary, forward-looking remarks, or a bibliography.
 - ↪ Assume the reader is already familiar with the broader comparison
 - ↪ between C++ templates and Java/C# generics. Begin directly with
 - ↪ the technical analysis.
- * Technical Demonstration: Provide a concise, valid Java code example
 - ↪ demonstrating the maximum achievable compile-time constraint
 - ↪ mechanism in Java, specifically bounded polymorphism (e.g., `<T`
 - ↪ `Number extends>`).
- * Theoretical Explanation: Detail exactly why full compile-time
 - ↪ branching on generic types (the equivalent of C++ Substitution
 - ↪ Failure Is Not An Error) fails in Java. Connect this limitation
 - ↪ mechanically and theoretically to type erasure.
- * Terminology: Use precise technical terminology appropriate for
 - ↪ programming language theory (e.g., type erasure, bounded
 - ↪ polymorphism, monomorphization, compile-time resolution, bounds
 - ↪ checking) where directly relevant.
- * Scope Control: Address only the replication of SFINAE/type-traits
 - ↪ in Java. Do not digress into memory footprint, hardware
 - ↪ performance, C# mechanisms, code bloat, garbage collection, or
 - ↪ unrelated language comparisons.

Writing Style Constraint:

- * Maintain a formal, objective, and highly precise academic writing
 - ↪ style.
- * Enforce strict conciseness: the section must be as short as fully
 - ↪ answering the prompt allows. Avoid padding, restating the
 - ↪ question, or using redundant transitions.

- * Clearly distinguish between compile-time mechanisms (bounds checking) and runtime realities (erasure) without confusing the two.
- * Do not use first-person pronouns (e.g., "I", "we", "our").

Output Structure:

Output only the raw technical text of the section. Use standard
 ↳ Markdown for formatting and code blocks. Do not include any
 ↳ overarching titles, placeholders, meta-commentary, or structural
 ↳ elements outside of what is strictly necessary to present the
 ↳ required technical content.

Sources:

- [7] provides the theoretical foundation of Java Generics, including the type erasure implementation model, the separation between compile-time generic type checking and runtime representation, and the limitations of Java parametric polymorphism with respect to type-dependent specialization.
- [19] provides the formal specification of Java bounded type parameters, including the semantics of `extends` bounds, compile-time type checking rules for generic classes and methods, and the erasure process applied to generic type parameters.
- [62] provides the theoretical background for C++ template metaprogramming, including template substitution, SFINAE-based overload resolution, type traits, and compile-time specialization mechanisms used as a comparison with the limitations of Java Generics.

2.22 Prompt 20: Limits of C# Generics in Replicating C++ SFINAE and Type Traits

goal: Generate a detailed academic section of the limits of Java generics in replicating C++ SFINAE and type traits. Show what's achievable using generic constraints (where `T : ...`) and, if relevant, .NET generic math / static abstract interface members. Explain why C# gets closer to C++ than Java does, but still can't do arbitrary compile-time computation or true SFINAE, tying it to reification vs monomorphization. **Model:** ChatGPT-5.5 **prompt:**

System Persona:

Act as an expert academic writer and programming language theorist
 ↳ specializing in the comparative analysis of parametric
 ↳ polymorphism and metaprogramming paradigms.

Goal:

Generate a single, concise, and highly technical section for a larger
→ academic chapter. This section must specifically analyze how C\#
→ approximates C++ SFINAE and type-traits using generic constraints
→ and generic math, while explaining its theoretical limitations.

Requirements:

* **Section-Level Framing (STRICT):** This text is an intermediate
→ section within an existing chapter. Do NOT include an
→ introduction, motivation, or structural framing (e.g., "In this
→ section, we will..."). Do NOT include a conclusion, summary, or
→ forward-looking transition. Do NOT include a bibliography or list
→ of references. Assume the reader already possesses full context
→ regarding the fundamental differences between C++ templates, Java
→ generics, and C\# generics.

* **Technical Execution:**

* Replicate the conceptual equivalent of a C++ SFINAE/type-traits
→ example using C\#.

* Demonstrate achievable compile-time/type-check-time constraints
→ using C\# bounded polymorphism (`where T : ...`).

* Illustrate the application of .NET generic math and static abstract
→ interface members to achieve trait-like capabilities.

* **Theoretical Analysis:**

* Explain why these C\# mechanisms allow it to approach C++
→ capabilities much more closely than Java does.

* Explain why C\# inherently cannot support true SFINAE (Substitution
→ Failure Is Not An Error) or arbitrary compile-time computation.

* Directly tie this limitation to the architectural differences
→ between .NET's runtime reification of generics and C++'s
→ compile-time monomorphization and AST-level substitution.

Writing Style Constraint:

* Maintain a formal, objective, and precise academic tone consistent
→ with programming language theory literature.

* Use precise terminology (e.g., bounded polymorphism, reification,
→ monomorphization, substitution failure) strictly where
→ applicable.

* Clearly distinguish between compile-time validation mechanisms and
→ runtime behavior.

- * Ensure strict scope control: address solely the requested C\#
 - ↪ replication of SFINAE/traits, generic constraints, generic math,
 - ↪ and the reification vs. monomorphization dichotomy. Do not
 - ↪ digress into memory footprint, code bloat, execution performance,
 - ↪ garbage collection, or unrelated language features.
- * Enforce absolute conciseness: provide only the information required
 - ↪ to fulfill the prompt. Omit conversational padding, restatements
 - ↪ of the prompt, and redundant transitional clauses.

Output Structure:

Provide only the finalized text and code blocks for this specific section. Do not output overarching titles, introductory meta-commentary, conversational filler, or any text outside the bounds of the academic section itself.

Sources:

- [37] The Microsoft documentation on C# generics is used for the description of generic constraints, bounded polymorphism through the ‘where’ clause, and the compile-time validation performed by the C# type system when generic arguments are instantiated.
- [36] The Microsoft documentation on .NET Generic Math is used for the analysis of static abstract interface members and interfaces such as ‘INumber<T>’, which provide trait-like abstractions for expressing type capabilities in generic algorithms.
- [62] The book *C++ Templates: The Complete Guide* is used for the comparison with C++ template metaprogramming, specifically regarding SFINAE, type traits, substitution failure, and compile-time template instantiation through monomorphization.
- [14] The ECMA-335 specification is used for the description of the CLR execution model, including the representation of generic types through runtime reification and the role of metadata in preserving generic type information.

2.23 Prompt 21: instantiation-time vs declaration-time checking

goal: Generate a detailed academic section to explain how the difference between C++ templates (checked only at instantiation) and Java/C# generics (checked at declaration) directly causes specific metaprogramming techniques to be impossible in Java/C#. **Model:** ChatGPT-5.5 **prompt:**

System Persona:

Act as a technical author and programming language theorist writing a
→ highly specialized section for an academic chapter on programming
→ language theory and software engineering.

Goal:

Generate a single, self-contained section of a larger chapter that
→ explains how the difference between C++ templates (checked only
→ at instantiation) and Java/C\# generics (checked at declaration)
→ directly causes specific metaprogramming techniques to be
→ impossible in Java/C\#. You must detail at least one concrete
→ technique that fails specifically due to this difference.

Requirements:

- * Focus strictly on the mechanisms of type checking:
 - instantiation-time (C++) versus declaration-time (Java/C\#).
 - * Explain the direct causal relationship between these checking mechanisms and the limitations placed on metaprogramming in Java and C\#.
 -
 - * Detail at least one concrete metaprogramming technique (e.g., SFINAE, or structural/duck typing within templates) that is possible in C++ but impossible in Java/C\# because of this specific constraint.
 -
 - * Include concise, precise code examples to illustrate the technique
 - in C++ and explicitly demonstrate why declaration-time checking prevents its equivalent in Java/C\#.
 - * Section-Level Framing (STRICT): Your output is strictly ONE SECTION
 - of an existing, larger chapter.
 - * Omit any standalone introduction, motivation, or "in this section we will" framing. Begin directly with the technical content.
 - * Omit any concluding paragraphs, chapter summaries, or forward-looking transitions.
 - * Omit a bibliography or reference list. Assume citations are handled globally at the end of the full chapter.
 - * Assume the reader already has the broader context of C++ templates versus Java/C\# generics. Do not re-introduce basic concepts.
-
- * Content Constraints:
 - * Use precise terminology (e.g., instantiation, substitution failure, bounded polymorphism, type erasure, reification) where strictly relevant to the prompt.
 - * Clearly distinguish between compile-time and runtime mechanisms as they relate to these type-checking models.

- * Enforce strict scope control. Do not digress into memory footprint,
 - ↪ hardware performance benchmarking, code bloat, garbage collection
 - ↪ overhead, or unrelated language comparisons. Address only the
 - ↪ type-checking differences and the resulting metaprogramming
 - ↪ limitations.

Writing Style Constraint:

- * Maintain a formal, objective, and precise technical writing style
 - ↪ consistent with programming language theory literature.
- * Enforce absolute conciseness: the section must be exactly as short
 - ↪ as necessary to fully answer the prompt. Include no padding, do
 - ↪ not restate the prompt, and avoid redundant transitions between
 - ↪ paragraphs.
- * Avoid unsupported claims; state assumptions about language or
 - ↪ compiler versions only if strictly necessary to explain the
 - ↪ technique.

Output Structure:

- Return only the raw text of the generated section, formatted using
- ↪ standard Markdown. Do not include any meta-commentary,
 - ↪ introductory remarks, conversational filler, or placeholders.
 - ↪ Begin immediately with the first sentence of the section's
 - ↪ technical content.

Sources:

- [62] This reference is used as the primary source for the C++ template instantiation model, including delayed type checking of dependent expressions, template substitution, and SFINAE-based overload resolution. The description of compile-time capability detection through substitution failure and structural requirements in templates is based on the template metaprogramming mechanisms discussed in this work.
- [5] This reference provides the theoretical background for Java generics, particularly the declaration-time type checking model, bounded polymorphism, and the requirement that all operations performed on a generic type parameter must be justified by its declared bounds. It supports the comparison between Java generic constraints and the structural flexibility provided by C++ template substitution.
- [14] This reference is used for the C# generic type system through the Common Language Infrastructure specification. It provides the formal description of generic parameters, constraints, and compile-time verification rules that restrict generic code to operations guaranteed by declared constraints rather than properties discovered during instantiation.

2.24 Prompt 22: External Metaprogramming Workarounds in Java and C#

Goal: Generate a detailed academic section of the external metaprogramming workarounds in Java and C#. Explain why these are workarounds external to the generics system rather than native metaprogramming. **Model:** ChatGPT-5.5
Prompt:

System Persona:

Act as an academic technical writer specializing in programming
→ language theory and software architecture.

Goal:

Write exactly ONE SECTION of an existing comprehensive chapter on
→ parametric polymorphism and metaprogramming. This specific
→ section must describe the common workarounds used in Java and C#
→ to simulate the native metaprogramming capabilities of C++
→ templates, and architecturally explain why these approaches are
→ external to their respective generics systems rather than native
→ metaprogramming.

Requirements:

- * Task 1: Technical Content Specification
 - * Detail Java's common workarounds: the use of runtime reflection,
→ compile-time annotation processors, and code generation utilities
→ (e.g., Lombok).
 - * Detail C#'s common workarounds: the use of source generators and
→ T4 templates.
 - * Articulate the structural and theoretical reasons why these
→ mechanisms are considered external workarounds layered over or
→ completely separate from the Java and C# generics systems,
→ contrasting them with the native, intrinsic compile-time
→ evaluation of C++ templates.
 - * Clearly differentiate between mechanisms that operate at runtime
→ versus those that operate at compile-time or pre-compilation.
- * Task 2: Section-Level Framing
 - * Output ONLY the body text for this specific section.
 - * Omit any standalone introduction, motivation, or prefatory remarks.
→ Do not use phrases such as "In this section we will examine...".
 - * Omit any conclusion, summary, or forward-looking remarks to
→ subsequent sections.
 - * Omit bibliographies or reference lists entirely.

- * Assume the reader already possesses full theoretical context
 - ↪ regarding C++ templates, Java type erasure, and C\# reified
 - ↪ generics. Begin directly with the technical exposition of the
 - ↪ workarounds.

- * Task 3: Content Constraints
- * Maintain a formal, objective, and precise technical writing style
 - ↪ consistent with programming language theory literature.
- * Use accurate terminology (e.g., compile-time, runtime, reflection,
 - ↪ AST manipulation, macro expansion) strictly where relevant.
- * Include brief, illustrative code snippets only if strictly
 - ↪ necessary to demonstrate a specific structural workaround
 - ↪ mechanism; otherwise, rely on rigorous theoretical explanation.
- * Strictly enforce scope control: address only the workarounds and
 - ↪ their relationship to the generics system. Do not digress into
 - ↪ memory footprint, hardware performance benchmarking, garbage
 - ↪ collection overhead, or unrelated architectural comparisons.
- * Enforce absolute conciseness: ensure the section is as short as
 - ↪ comprehensively answering the prompt allows. Avoid padding,
 - ↪ redundant paragraph transitions, or restating the prompt's
 - ↪ premises.

Writing Style Constraint:

Do not include any meta-commentary, explanations, placeholders, or

- ↪ out-of-character text. Do not use first-person pronouns ("I",
- ↪ "we"). Produce only the final text of the requested section.

Output Structure:

Return the technical text of the section directly. Use standard

- ↪ Markdown for logical structuring (such as subheadings for
- ↪ language-specific mechanisms), but do not include a top-level
- ↪ chapter title.

Sources:

- [43] is used as a reference for: Java annotation processing, compiler-based analysis of source elements, and compile-time generation of additional source files. The same reference is also used to describe Java reflection as a runtime mechanism based on class metadata inspection.
- [48] is used as a reference for: Java code generation through compiler extensions, source transformation techniques, and the generation of program elements outside the Java generics mechanism.

- [39] is used as a reference for: C# Source Generators based on Roslyn, T4 Text Templates, and their role as external code generation mechanisms separated from the C# generic type system.
- [59] is used as a reference for: the C++ template instantiation model, compile-time substitution of template parameters, and the integration of metaprogramming capabilities directly into the language construct.

Bibliography

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson, 3 edition, 2022.
- [2] Andrew W. Appel. *Modern Compiler Implementation in C*. Cambridge University Press, 1998.
- [3] Joshua Bloch. *Effective Java*. Addison-Wesley Professional, 3 edition, 2018.
- [4] Gilad Bracha. Generics in the java programming language, 2004.
- [5] Gilad Bracha. Generics in the java programming language. Technical report, Sun Microsystems, 2004.
- [6] Gilad Bracha, Martin Odersky, David Stoutamire, and Philip Wadler. Making the future safe for the past: Adding genericity to the java programming language. In *Proceedings of the 13th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 183–200. ACM, 1998.
- [7] Gilad Bracha, Martin Odersky, David Stoutamire, and Philip Wadler. Making the future safe for the past: Adding genericity to the java programming language. In *Proceedings of the ACM Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. ACM, 1998.
- [8] Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys*, 17(4):471–523, 1985.
- [9] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. Morgan Kaufmann, 3 edition, 2022.
- [10] cppreference.com. *C++ Language Reference: RTTI and typeid*. cppreference.com, 2025. Online reference for C++ RTTI mechanisms, `std::type_info`, `typeid`, and `dynamic_cast`.
- [11] cppreference.com. `constexpr` specifier. <https://en.cppreference.com/w/cpp/language/constexpr>, 2025.
- [12] cppreference.com. Templates. <https://en.cppreference.com/w/cpp/language/templates>, 2025.
- [13] Ecma International. Ecma-334: C# language specification, 2023.

- [14] Ecma International. Ecma-335: Common language infrastructure (cli), 2024.
- [15] Margaret A. Ellis and Bjarne Stroustrup. *The Annotated C++ Reference Manual*. Addison-Wesley Professional, Reading, MA, 1990.
- [16] GNU Project. G++ implementation details, 2024. Official GCC documentation describing C++ implementation mechanisms.
- [17] Brian Goetz, Tim Peierls, Joshua Bloch, Joseph Bowbeer, David Holmes, and Doug Lea. *Java Concurrency in Practice*. Addison-Wesley Professional, 2006.
- [18] James Gosling, Bill Joy, Guy L. Steele Jr., Gilad Bracha, and Alex Buckley. *The Java Language Specification*. Oracle, java se 21 edition, 2023.
- [19] James Gosling, Bill Joy, Guy Steele, Gilad Bracha, and Alex Buckley. *The Java Language Specification*. Addison-Wesley, 8th edition, 2014.
- [20] James Gosling, Bill Joy, Guy Steele, Gilad Bracha, and Alex Buckley. *The Java Language Specification, Java SE 8 Edition*. Oracle, 2014.
- [21] James Gosling, Bill Joy, Guy Steele, Gilad Bracha, and Alex Buckley. *The Java Language Specification*. Oracle, java se 21 edition, 2023.
- [22] Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. Featherweight java: A minimal core calculus for java and gj. *ACM Transactions on Programming Languages and Systems*, 23(3):396–450, 2001.
- [23] ISO/IEC. *ISO/IEC 14882:2020 Programming Languages – C++*. International Organization for Standardization, 2020.
- [24] ISO/IEC. *Programming languages – c++*, 2024.
- [25] ISO/IEC JTC 1/SC 22/WG 21. *Programming languages — c++*. Technical Report ISO/IEC 14882:2020, International Organization for Standardization, 2020.
- [26] ISO/IEC JTC 1/SC 22/WG 21. *ISO/IEC 14882:2024 Programming Languages — C++*, 2024. International Standard.
- [27] Itanium C++ ABI Project. Itanium c++ abi: C++ abi specification, 2024. C++ Application Binary Interface specification including name mangling and RTTI representation.
- [28] James Gosling and Bill Joy and Guy Steele and Gilad Bracha and Alex Buckley. *The java language specification*. Technical report, Oracle, 2023.
- [29] Andrew Kennedy and Don Syme. Design and implementation of generics for the .net common language runtime. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 1–12. ACM, 2001.

- [30] Tim Lindholm, Frank Yellin, Gilad Bracha, Alex Buckley, Daniel Smith, and Gavin Bierman. *The Java Virtual Machine Specification*. Oracle, java se 21 edition, 2023.
- [31] Stanley B. Lippman. *Inside the C++ Object Model*. Addison-Wesley Professional, Reading, MA, 1996.
- [32] LLVM Project. Clang internals manual, 2024. Compiler implementation details for C++ semantic analysis and code generation.
- [33] LLVM Project. Llvm language reference manual, 2024. LLVM intermediate representation and code generation model.
- [34] John McCall. *Generics in swift: Implementation and runtime representation*, 2019.
- [35] Microsoft. *C++ Templates Documentation*. Microsoft Learn, 2024.
- [36] Microsoft. Generic math in .net. <https://learn.microsoft.com/en-us/dotnet/standard/generics/math>, 2025.
- [37] Microsoft. Generics in c#. <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/generics>, 2025.
- [38] Microsoft Corporation. *C# Language Specification*. Microsoft, 2023.
- [39] Microsoft Corporation. Code generation in c#: Source generators and t4 text templates. <https://learn.microsoft.com/dotnet/csharp/roslyn-sdk/source-generators-overview>, 2024.
- [40] Steven S. Muchnick. *Advanced Compiler Design and Implementation*. Morgan Kaufmann, 1997.
- [41] Oracle. The java language specification, java se 24 edition, 2024.
- [42] Oracle. The java virtual machine specification, java se 24 edition, 2024.
- [43] Oracle Corporation. Java compiler api and annotation processing. <https://docs.oracle.com/javase/>, 2024.
- [44] Oracle Corporation. The java virtual machine specification. [<https://docs.oracle.com/javase/specs/jvms/>] (<https://docs.oracle.com/javase/specs/jvms/>), 2025.
- [45] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, Cambridge, MA, 2002.
- [46] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [47] Benjamin C. Pierce, editor. *Advanced Topics in Types and Programming Languages*. MIT Press, Cambridge, MA, 2005.

- [48] Project Lombok. Project lombok: Spice up your java. <https://projectlombok.org/>, 2024.
- [49] Jeffrey Richter. *CLR via C#*. Microsoft Press, 4 edition, 2012.
- [50] Jeffrey Richter. *CLR via C#: A Developer's Tour of the Common Language Runtime*. Microsoft Press, 4th edition, 2012.
- [51] Christopher Strachey. Fundamental concepts in programming languages, 1967. Lecture Notes, International Summer School in Computer Programming, Copenhagen.
- [52] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley Professional, 4 edition, 2013.
- [53] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley Professional, 4th edition, 2013.
- [54] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley Professional, 4 edition, 2013.
- [55] The Rust Project Developers. Monomorphization and code generation, 2024.
- [56] The Rust Project Developers. The rust reference, 2024. Official Rust language specification.
- [57] David Vandevor, Nicolai M. Josuttis, and Douglas Gregor. *C++ Templates: The Complete Guide*. Addison-Wesley Professional, 2 edition, 2017.
- [58] David Vandevor, Nicolai M. Josuttis, and Douglas Gregor. *C++ Templates: The Complete Guide*. Addison-Wesley Professional, 2nd edition, 2017.
- [59] David Vandevor, Nicolai M. Josuttis, and Douglas Gregor. *C++ Templates: The Complete Guide*. Addison-Wesley Professional, 2 edition, 2017.
- [60] David Vandevor, Nicolai M. Josuttis, and Douglas Gregor. *C++ Templates: The Complete Guide*. Addison-Wesley Professional, 2nd edition, 2017.
- [61] David Vandevor, Nicolai M. Josuttis, and Douglas Gregor. *C++ Templates: The Complete Guide*. Addison-Wesley Professional, 2 edition, 2017.
- [62] David Vandevor, Nicolai M. Josuttis, and Douglas Gregor. *C++ Templates: The Complete Guide*. Addison-Wesley Professional, 2nd edition, 2017.
- [63] Bill Venner. *Inside the Java Virtual Machine*. McGraw-Hill, 2 edition, 2000.
- [64] Richard Warburton. Java generics and type inference, 2014. Oracle Technical Documentation.